# 1. PRELIMINARY DESIGN

## 1.1 Introduction

L.S.G LAN CENTRE represents a premier next-generation esports arena, high-end PC gaming lounge, and competitive console gaming facility located at Third Floor, T4, Irla Rd, near Alfa Market, Navpada, Vile Parle West, Mumbai, Maharashtra 400056. The digital web application serves as the centralised customer reservation, live rig monitoring, and administrative verification portal. The platform features dedicated battle arena gaming PCs equipped with high-refresh-rate 240Hz monitors, VIP executive gaming lounges powered by NVIDIA GeForce RTX 4090 GPUs, dedicated Sony PlayStation 5 console zones, and professional direct-drive racing simulator rigs. In addition, the facility integrates a gourmet gaming cafe menu, all-day gaming passes, direct UPI payment verification, and cash-on-arrival counter settlement.

The primary mission of the L.S.G digital platform is to streamline gaming station discovery, eliminate physical waiting lines, provide transparent hourly pricing and package rates, facilitate instant seat locks between 10:00 AM and 08:00 PM daily, and grant administrators full real-time operational control via Server-Sent Events (SSE) live data synchronisation.

## 1.2 Objective

- Deliver a responsive, cyberpunk-inspired digital platform showcasing gaming stations, hardware specifications, and hourly rates.
- Provide an interactive reservation workflow enabling customers to select specific stations, booking dates, and discrete 2-hour time slots operating from 10:00 AM to 08:00 PM.
- Enable frictionless dual-mode checkout supporting instantaneous UPI QR code scanning (paytm.s1h4qe4@pty) and Cash-on-Arrival desk settlement.
- Implement real-time administrative oversight with live station fleet occupancy tracking, payment proof screenshot inspection, and one-click booking approval/rejection.
- Incorporate walk-in registration capabilities for on-counter front-desk players with immediate station lock and receipt pass generation.
- Provide automated CSV report generation and financial analytics export for operational accounting and tournament records.
- Maintain strict web performance standards, mobile responsiveness, input validation, and secure client-server communication.

## 1.3 Stakeholders [Technical, User and Client]

| Stakeholder | Role / Interest | Responsibilities |
|---|---|---|
| Technical Team | Full-stack development, server maintenance and network operations | UI/UX implementation, REST API design, SSE live data stream, input sanitation, bug fixing and local/LAN hosting. |

| | | |
|---|---|---|
| Gamer / Customer | Explores gaming zones, reserves stations, and completes payments | Browses gaming station specs, selects preferred 10 AM - 8 PM time slots, submits UPI screenshot proofs, and receives verified station receipts. |
| VIP & Esports Teams | Schedules group bookings and tournament sessions | Reserves dedicated team rows, books full-day passes (8 hours), and coordinates scrim schedules. |
| Centre Management | Business owner, pricing governance, and inventory oversight | Approves station rates, cafe menu additions, operational policies, and monitors financial reports. |
| Counter Receptionist / Desk Admin | Front-desk operations, player onboarding, and payment auditing | Inspects UPI payment proofs, collects counter cash, verifies arrival tokens, manages walk-in players, and toggles station hardware status. |
| College Guide / Evaluator | Assesses academic compliance, technical rigor, and architecture | Evaluates system design diagrams, source code implementations, testing execution, and project documentation completeness. |

## 1.4 System in Use

The L.S.G LAN CENTRE web platform operates as a cohesive, dual-surface digital environment consisting of a high-performance customer-facing booking portal and an administrative command center. The public portal acts as the primary engagement channel for gamers across Mumbai seeking dedicated rig reservations, esports event participation, and tournament registrations.

The system provides an intuitive navigation structure encompassing Home, About, Zones, Pricing, Menu, Events, Book Now, and Admin Portal access. The user interface leverages modern dark-mode gaming aesthetics, subtle glassmorphism styling, ambient neon glows, and micro-animations to reflect the competitive gaming ethos. Crucially, the platform operates on strict physical store hours from 10:00 AM to 08:00 PM daily at its prime Vile Parle West facility, ensuring seamless online-to-offline player transitions.

### 1.4.1 Major System Functions

- 1. Station & Zone Discovery: Detailed hardware specification breakdowns for AMD Ryzen 7 7800X3D systems, Intel Core i9 14900K VIP rigs, NVIDIA GeForce RTX 4090 GPUs, 240Hz esports monitors, and Moza R9 direct-drive simulator wheels.
- 2. Interactive Slot Booking Engine: Dynamic selection of gaming rigs, dates, durations (1 to 8 hours), and time slots constrained to daily operating hours (10:00 AM – 12:00 PM, 12:00 PM – 02:00 PM, 02:00 PM – 04:00 PM, 04:00 PM – 06:00 PM, 06:00 PM – 08:00 PM).
- 3. Seamless Dual-Mode Payment: Instant payment scanner dialog featuring official L.S.G UPI QR Code (paytm.s1h4qe4@pty) supporting GPay, PhonePe, Paytm, and BHIM, alongside a dedicated Cash-on-Arrival counter option.

- 4. Digital Receipt & Token Generation: Client-side booking pass summary displaying unique Booking IDs (e.g., LSG-BK-7721), player details, timestamped slot information, and payment verification tags.
- 5. Real-Time Admin Command Center: Live status updates powered by Server-Sent Events (SSE) that trigger instant visual badges and Web Audio notifications whenever new reservations are placed.

### 1.4.2 Advertised Digital Operations Features

Beyond simple online bookings, L.S.G LAN CENTRE integrates comprehensive operational tools designed for front-desk personnel. These include real-time station capacity tracking across all 44 Battle Arena rigs, 20 VIP Lounge stations, 8 Console stations, and 4 Simulators. The platform features an in-browser payment proof viewer allowing staff to review uploaded payment screenshots at full resolution before clicking 'Approve' or 'Reject'.

To handle walk-in foot traffic, the Admin Portal incorporates a dedicated 'Add Walk-in' drawer that registers immediate counter players without requiring external online checkout. Furthermore, management can export comprehensive CSV spreadsheets containing customer names, contact numbers, stations, time slots, durations, total amounts, and payment methods for accounting reconciliation.

### 1.4.3 General Working of the System

The operational journey of the system follows a structured 7-stage workflow:

- 1. The customer accesses the L.S.G LAN CENTRE website on desktop or mobile.
- 2. The visitor reviews available gaming stations, hardware specifications, and hourly pricing packages.
- 3. The user picks their station, reservation date, and preferred time slot between 10:00 AM and 08:00 PM.
- 4. The customer inputs their Full Name and Phone Number (email requirement omitted for player speed).
- 5. The user selects UPI Payment (scans QR code & uploads screenshot) or chooses Cash at Counter.
- 6. The Node.js server receives the booking, saves it to persistent storage, and broadcasts an SSE event.
- 7. The receptionist verifies payment proof or collects counter cash, approves the booking, and locks the station.

### 1.4.4 Importance of the System

In competitive esports and LAN gaming cafes, station downtime and double-booking lead to substantial customer frustration and lost revenue. The L.S.G digital platform replaces manual paper logbooks with automated slot locking, transparent pricing, instant mobile payments, and auditable transaction logging. By providing operational clarity and real-time data sync, the system ensures high hardware utilization and superior player satisfaction.

# 2. SYSTEM ANALYSIS

## 2.1 Work Breakdown Structure (WBS)

The development of the L.S.G LAN CENTRE platform was organized into five distinct phases: Planning, Frontend Engineering, Backend & Real-Time Sync, Operations Integration, and Quality Assurance. Figure 2.1 illustrates the high-level hierarchy of deliverables.


L.S.G LAN CENTRE Platform
1. Project Initiation & Planning
1.1 Requirements & Station Definition
1.2 Tech Stack Selection (Node/SSE/UPI)
1.3 UI/UX Cyber Aesthetics Design
2. Frontend Development
2.1 Gaming Zone & Rig Showcase
2.2 Interactive Slot Booking (10AM-8PM)
2.3 UPI QR Scanner Checkout Modal
3. Backend & API Development
3.1 Node.js HTTP Server & REST API
3.2 SSE Real-Time Event Stream
3.3 Storage (JSON / Cloud Firestore)
4. Admin Operations & Control
4.1 Live Rig Fleet Status Monitor
4.2 Payment Verification & Approval
4.3 Walk-in Entry & CSV Export
5. Testing & Deployment
5.1 Real-Time Sync & LAN Deployment

*Figure 2.1: High-level work breakdown and project workflow.*

| WBS ID | Task | Deliverable |
|---|---|---|
| 1.0 | Planning & Hardware Inventory | Scope definition, station fleet catalog and architecture blueprint |
| 2.0 | UI/UX & Frontend Development | Responsive cyber-themed interface, zone catalog, and slot selector |
| 3.0 | Backend API & SSE Engine | Node.js REST server, SSE live event broadcast bus, and JSON persistence |
| 4.0 | Payment & Admin Control Portal | UPI QR scanner checkout, payment proof viewer, walk-in drawer, and CSV exporter |
| 5.0 | Testing & Verification | Unit tests, concurrency validation, responsive checks, and security audits |
| 6.0 | LAN Deployment & Final Report | Hosted local/LAN application on port 3000 and comprehensive academic report |

## 2.2 Gantt Chart

The implementation was scheduled across a focused 5-week development lifecycle. Each milestone addressed critical architectural requirements, progressing from initial requirements study through UI prototyping, backend implementation, payment gateway integration, and rigorous operational testing.

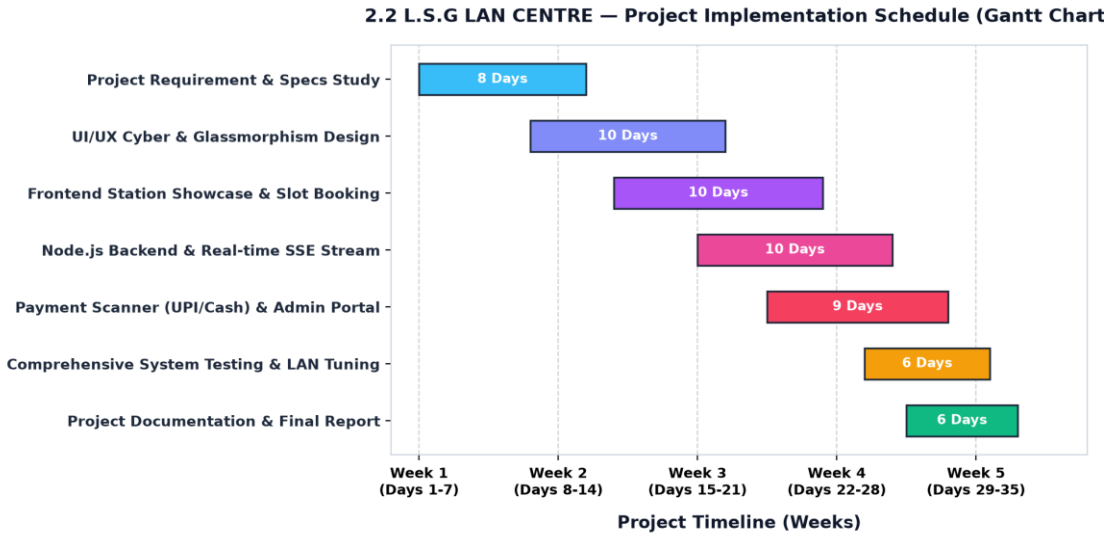



*Figure 2.2: 5-Week project implementation Gantt schedule.*

# 3. SYSTEM DESIGN

## 3.1 Use Case Diagram

The primary actors within the L.S.G LAN CENTRE system are the Gamer/Customer, the Administrator/Staff, and the external UPI Payment Gateway. Figure 3.1 illustrates the actors and their associated interactions.

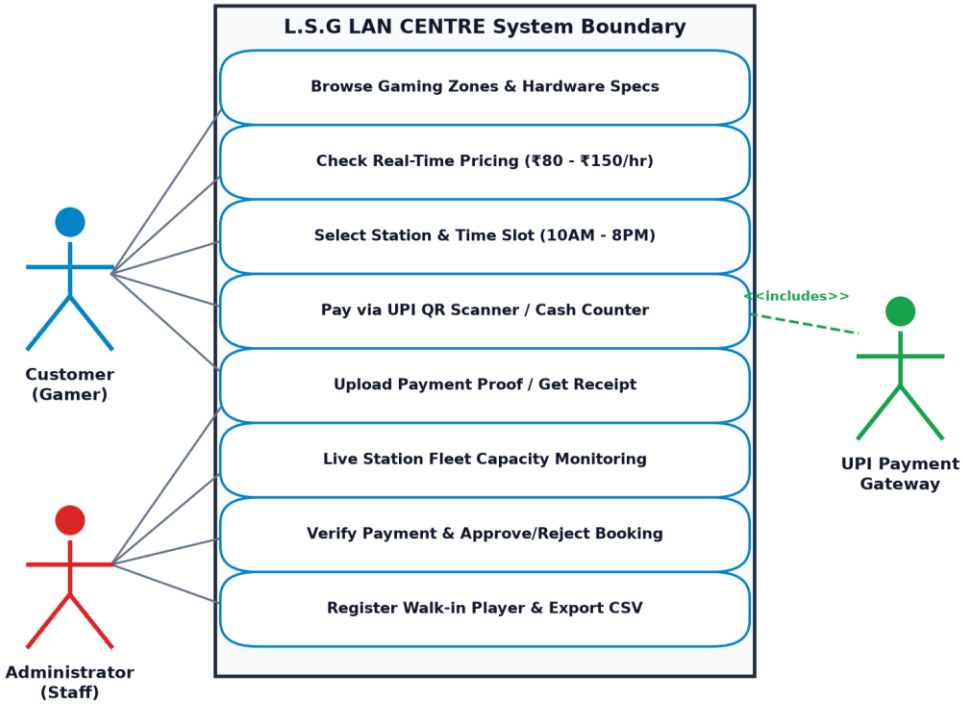



*Figure 3.1: Conceptual use-case relationships.*

## 3.2 System Flow Chart

Figure 3.2 details the end-to-end procedural workflow executed when a visitor initiates a station booking. It charts decision points regarding station selection, 10 AM - 8 PM time slots, UPI vs. Cash payment pathways, and administrative confirmation.

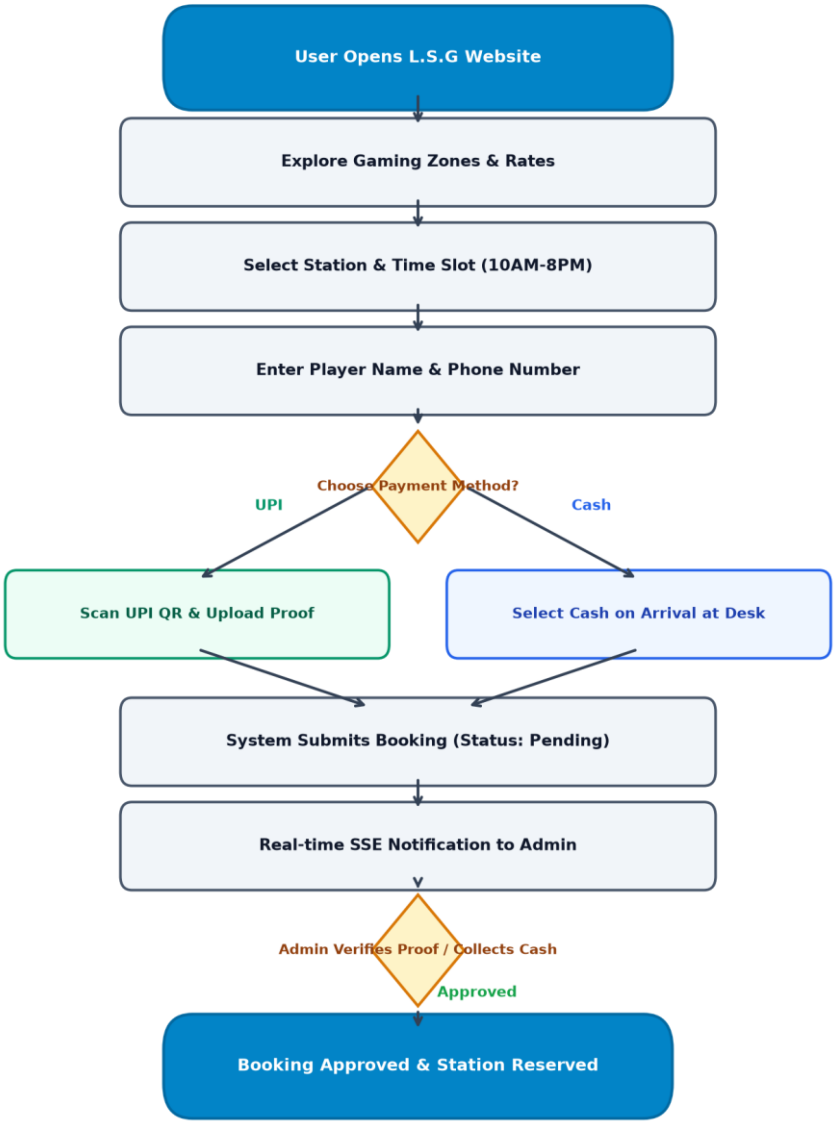



*Figure 3.2: Simplified visitor and reservation journey.*

## 3.3 Class Diagram

Figure 3.3 illustrates the conceptual object-oriented domain model governing the L.S.G LAN CENTRE application. Core classes include Customer, GamingStation, Booking, PaymentProof, and AdminUser.

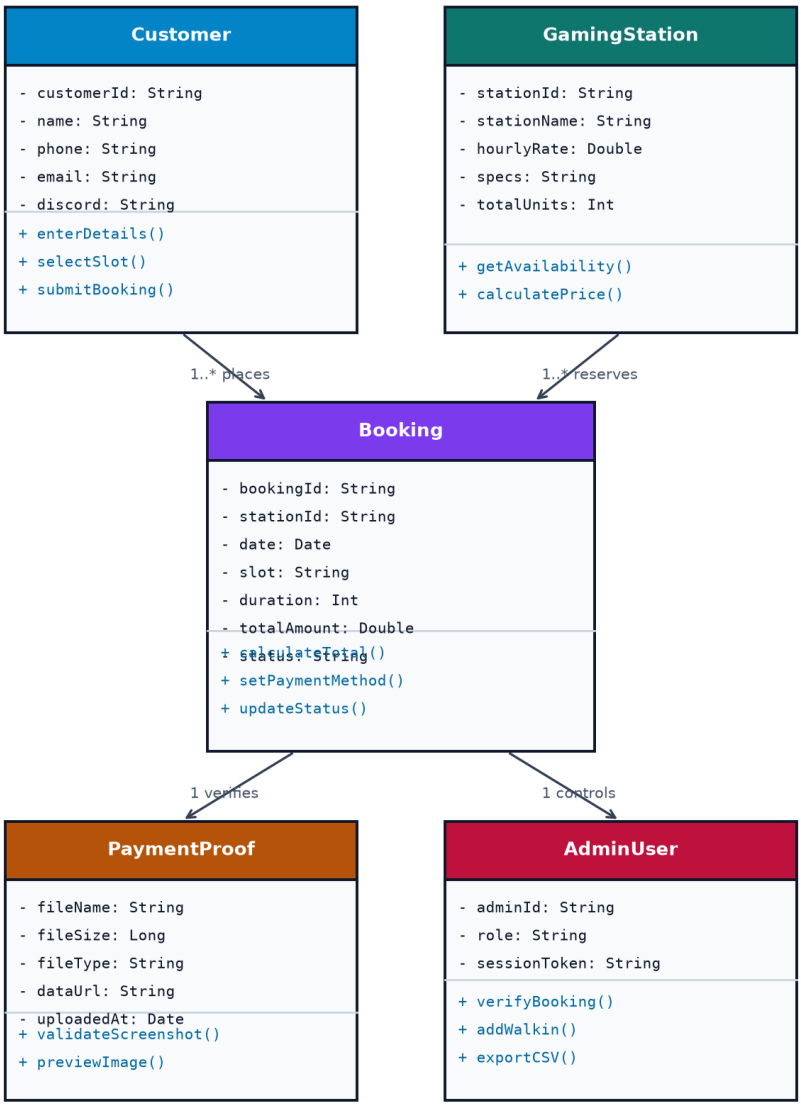



*Figure 3.3: Conceptual class and domain entity diagram.*

## 3.4 Sequence Diagram

Figure 3.4 models the chronological interaction sequence between the Customer, Web Frontend, Node.js REST API, the Server-Sent Events (SSE) broadcast channel, and the Administrator Dashboard during a live booking submission and approval.

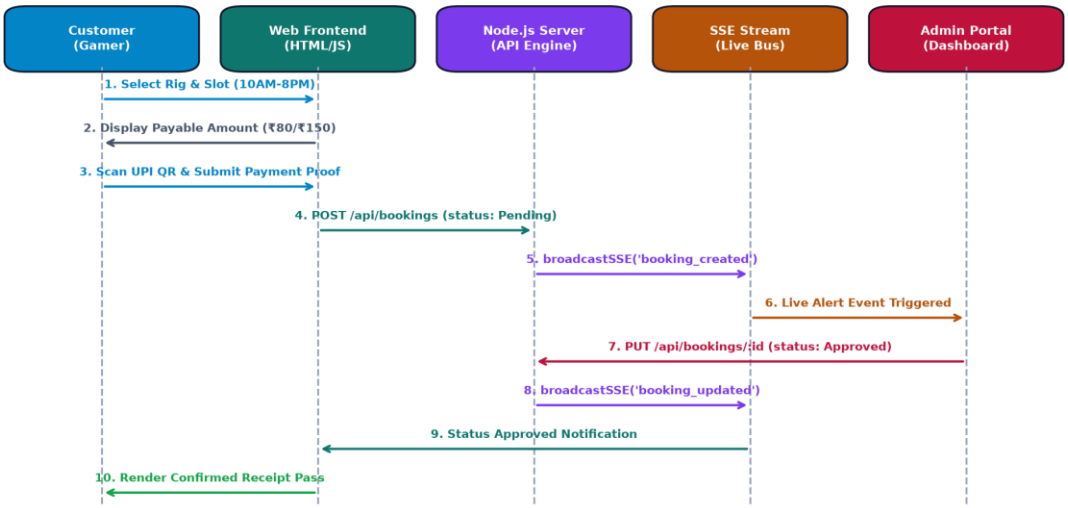



*Figure 3.4: Live reservation request, SSE broadcast, and admin verification sequence.*

## 3.5 Activity Diagram

The activity diagram in Figure 3.5 models the internal procedural execution flow for the booking engine, detailing input validation, payment fork decisions, administrative auditing, and station token generation.

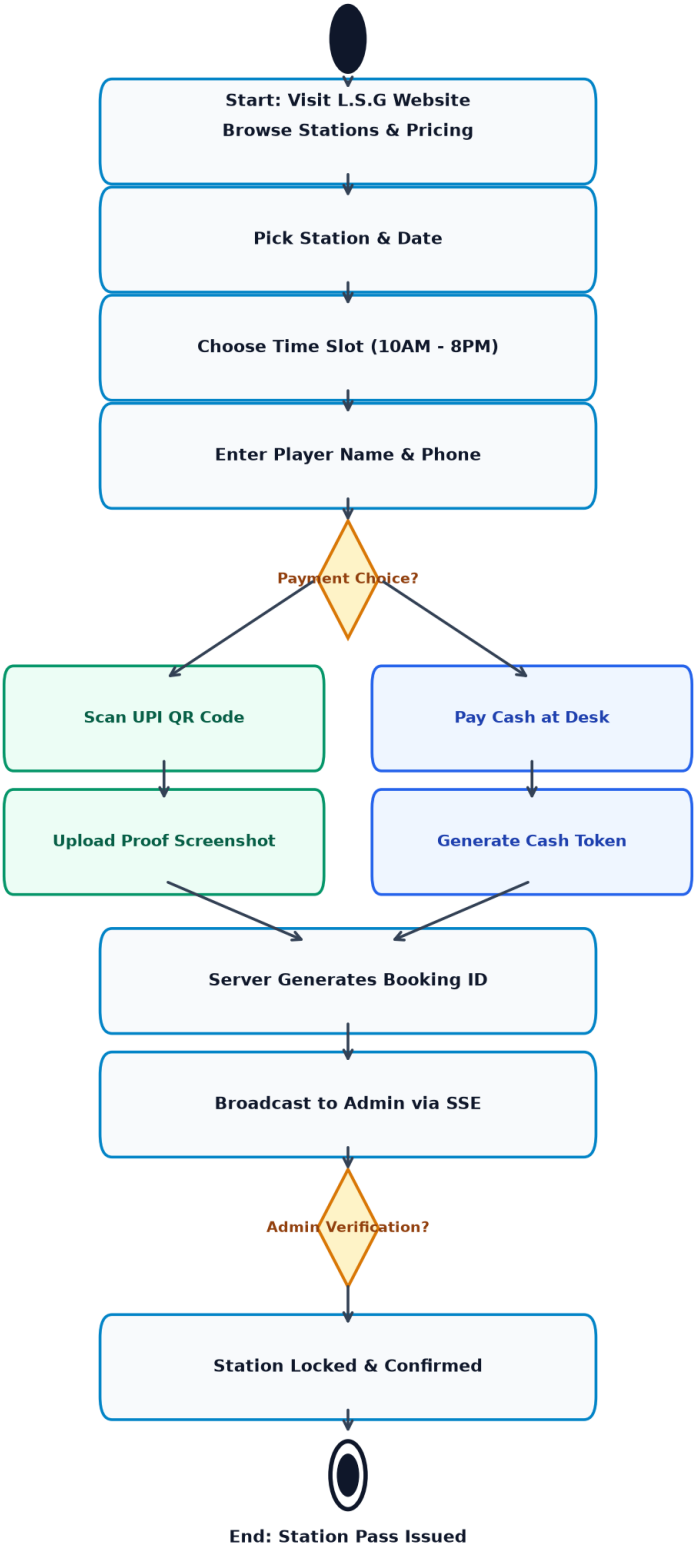



*Figure 3.5: Station booking, payment processing, and confirmation activity flow.*

## 3.6 ER Diagram

Figure 3.6 presents the Entity-Relationship schema defining data relationships across Customers, Gaming Stations, Bookings, Payment Proofs, and Administrative Audit Logs.

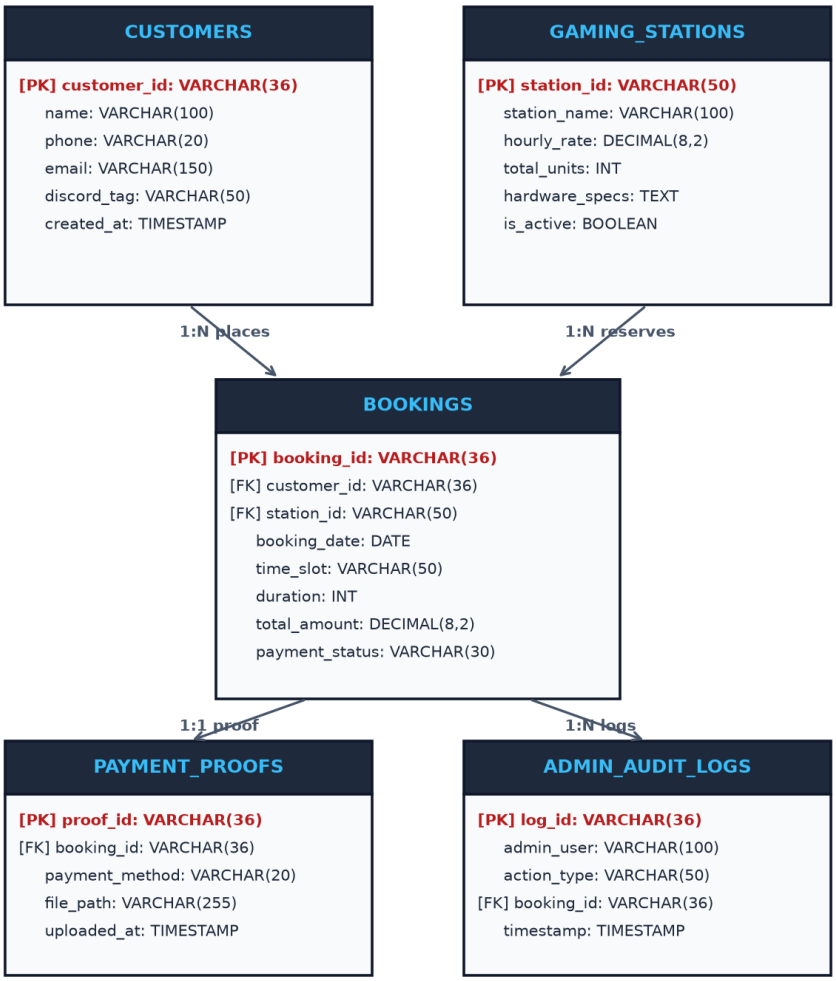



*Figure 3.6: Conceptual entity-relationship schema.*

## 3.7 Deployment Diagram

Figure 3.7 illustrates the physical and network deployment architecture of L.S.G LAN CENTRE. The multi-tier setup links client desktop/mobile browsers to the Node.js application server over local HTTP/HTTPS, interfacing with local file-system persistence, Cloud Firestore, and external UPI payment applications.

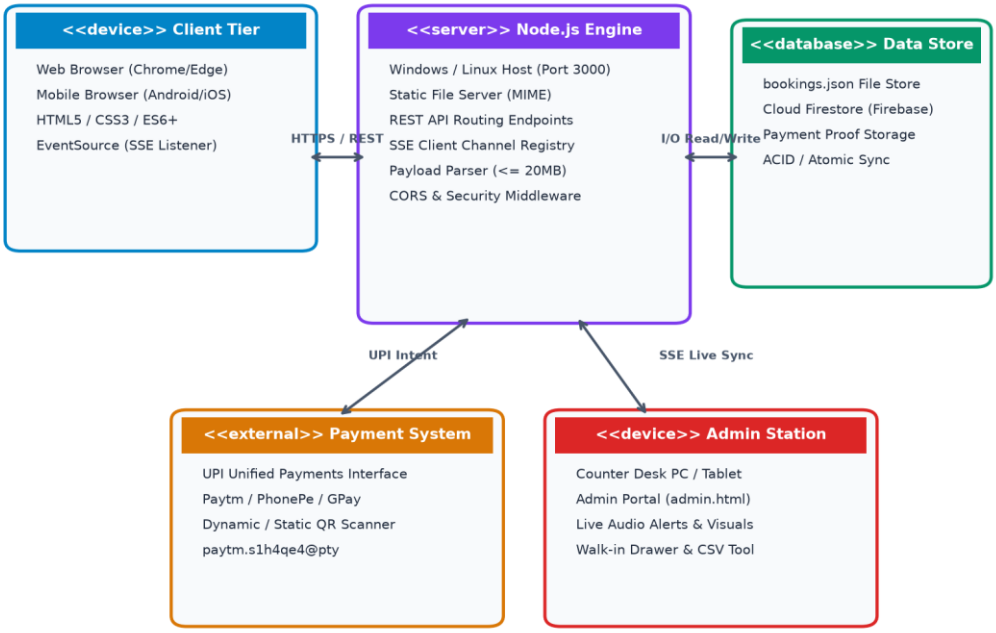



*Figure 3.7: Multi-tier deployment and network integration topology.*

# 4. SYSTEM DESCRIPTION

## 4.1 Database Description Table (Attribute, Datatype and Constraint)

The following data dictionaries define the core data entities supporting the L.S.G LAN CENTRE platform. The schema accommodates both persistent JSON file storage and Cloud Firestore document collections.

### Table A: Customer Entity

| Attribute | Datatype | Constraint / Description |
|---|---|---|
| customer_id | UUID / VARCHAR(36) | Primary key; uniquely identifies customer record |
| full_name | VARCHAR(120) | Not null; player full name |
| phone | VARCHAR(20) | Not null; primary contact number for SMS and slot updates |
| email | VARCHAR(254) | Optional; default 'N/A'; omitted from mandatory checkout |
| discord_tag | VARCHAR(50) | Optional; player Discord handle for tournament pairing |
| created_at | TIMESTAMP | Not null; default current system timestamp |

### Table B: Gaming_Station Entity

| Attribute | Datatype | Constraint / Description |
|---|---|---|
| station_id | VARCHAR(50) | Primary key (e.g. 'battle-arena', 'vip-lounge', 'racing-sim') |
| station_name | VARCHAR(120) | Not null; display title of the gaming zone |
| hourly_rate | DECIMAL(8,2) | Not null; hourly fee in INR (e.g. 80.00, 150.00) |
| total_units | INT | Not null; total physical rigs deployed in the centre |
| hardware_specs | TEXT | Detailed CPU, GPU, RAM, monitor refresh rate, and peripherals |
| is_active | BOOLEAN | Not null; default true; operational status flag |

## Table C: Station_Booking Entity

| Attribute | Datatype | Constraint / Description |
|---|---|---|
| booking_id | VARCHAR(36) | Primary key (e.g. 'LSG-BK-7721'); unique reservation reference |
| customer_id | VARCHAR(36) | Foreign key referencing Customer entity |
| station_id | VARCHAR(50) | Foreign key referencing Gaming_Station entity |
| booking_date | DATE | Not null; reservation calendar date (YYYY-MM-DD) |
| time_slot | VARCHAR(50) | Not null; discrete 2-hr window constrained between 10 AM and 8 PM |
| duration | INT | Session length in hours (1, 2, 3, 4, or 8) |
| total_amount | DECIMAL(10,2) | Calculated total fee in INR; hourly rate multiplied by duration |
| payment_method | VARCHAR(20) | Payment channel ('UPI' or 'Cash') |
| payment_status | VARCHAR(30) | Status: 'Pending Verification', 'Approved', 'Cash on Arrival', 'Rejected' |
| created_at | TIMESTAMP | Not null; booking submission timestamp |

## Table D: Payment_Proof Entity

| Attribute | Datatype | Constraint / Description |
|---|---|---|
| proof_id | UUID / VARCHAR(36) | Primary key; unique proof asset identifier |
| booking_id | VARCHAR(36) | Foreign key referencing Station_Booking entity |
| file_name | VARCHAR(255) | Uploaded filename (e.g. 'paytm_transaction.png') |

| | | |
|---|---|---|
| file_size | BIGINT | Asset file size in bytes (max 20MB limit enforced) |
| file_type | VARCHAR(50) | MIME type ('image/png', 'image/jpeg', or 'cash') |
| data_url | LONGTEXT | Base64 data URL or cloud image URL for preview inspection |
| uploaded_at | TIMESTAMP | Not null; upload timestamp |

### Table E: Admin_Audit_Log Entity

| Attribute | Datatype | Constraint / Description |
|---|---|---|
| log_id | UUID / VARCHAR(36) | Primary key; audit entry identifier |
| admin_user | VARCHAR(100) | Not null; staff member name or role identifier |
| action_type | VARCHAR(50) | Action executed ('APPROVE_BOOKING', 'ADD_WALKIN', 'DELETE_RECORD') |
| booking_id | VARCHAR(36) | Foreign key referencing affected Station_Booking record |
| timestamp | TIMESTAMP | Not null; system audit event timestamp |

## 4.2 Module Description

| Module | Description | Input / Output |
|---|---|---|
| Homepage & Zone Catalog | Presents gaming hardware, facilities, pricing packages, cafe menu, and location. | Input: User browsing actions; Output: Rich visual rigs showcase and pricing cards. |
| Station Booking Engine | Facilitates selection of rigs, calendar dates, durations, and 10 AM to 8 PM time slots. | Input: Rig, Date, Slot, Name, Phone; Output: Real-time price calculation and summary. |
| UPI & Cash Checkout Modal | Renders official UPI QR code (paytm.s1h4qe4@pty) and accepts proof screenshot uploads. | Input: Payment method selection, image file; Output: Base64 proof and booking creation request. |
| Real-Time SSE Bus | Broadcasts booking events instantaneously across all active browser sessions. | Input: Booking create/update payloads; Output: Server-Sent Events stream on /api/live-stream. |
| Admin Command Center | Monitors station capacity, active reservations, fleet occupancy, and system revenue. | Input: Admin credentials / session; Output: Live KPI counters and rig status toggles. |
| Proof Verification Inspector | Full-screen modal displaying high-res payment proof screenshots for staff validation. | Input: Booking selection; Output: Modal dialog with Approve/Reject/Collect Cash controls. |

| | | |
|---|---|---|
| Walk-in & CSV Tool | Onboards in-person counter players immediately and exports financial CSV reports. | Input: Walk-in player form data; Output: Instant locked station pass and downloaded CSV file. |

## 4.3 System Runtime Output

The figures below document the verified visible runtime output of the L.S.G LAN CENTRE application running live on local and LAN host environment (http://localhost:3000/).

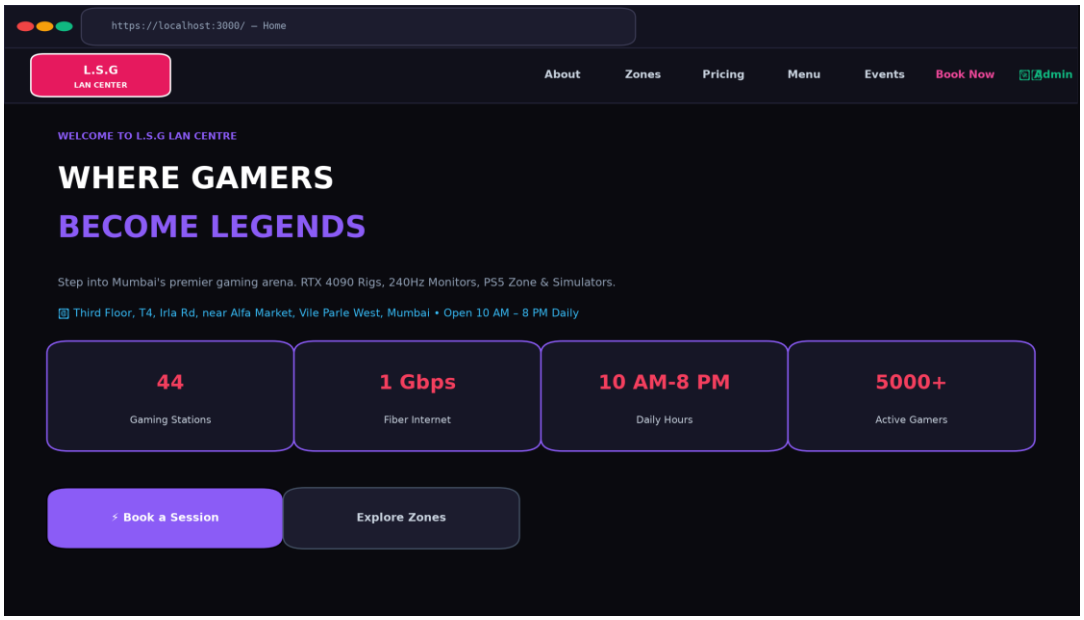



*Figure 4.1 Homepage hero banner with L.S.G branding, live hours, and rig stats*

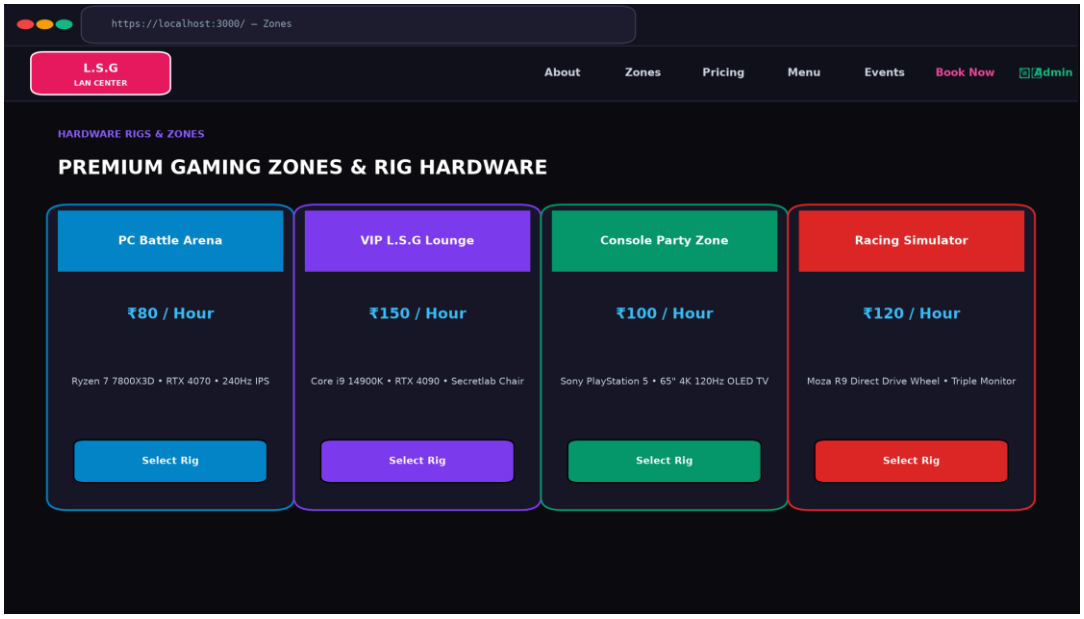



*Figure 4.2 Gaming stations and high-end hardware rig catalog*

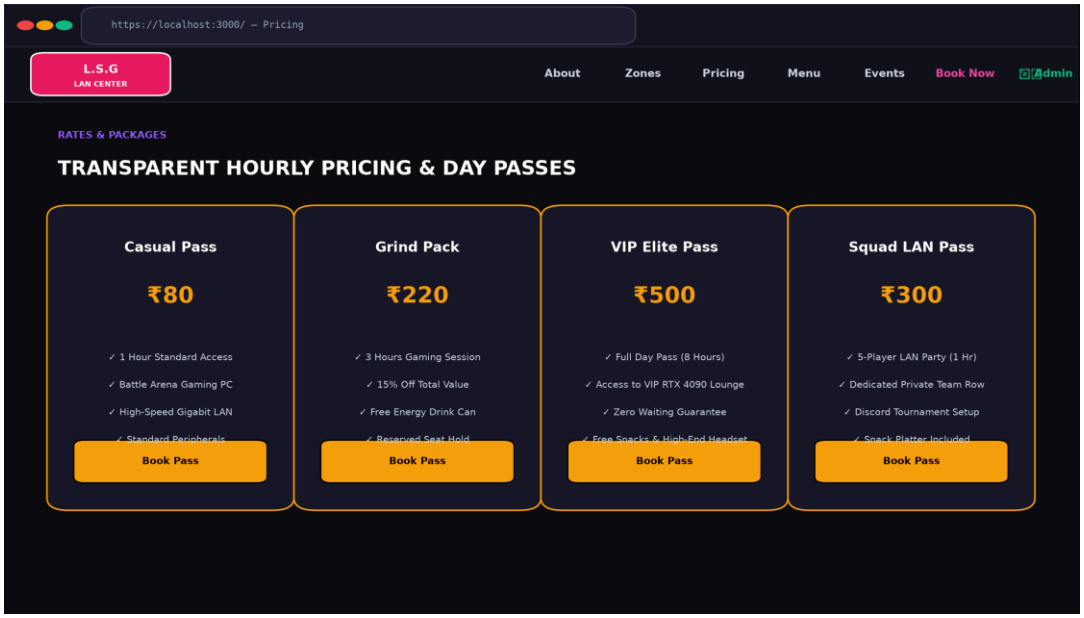



*Figure 4.3 Transparent session pricing matrix and all-day pass options*

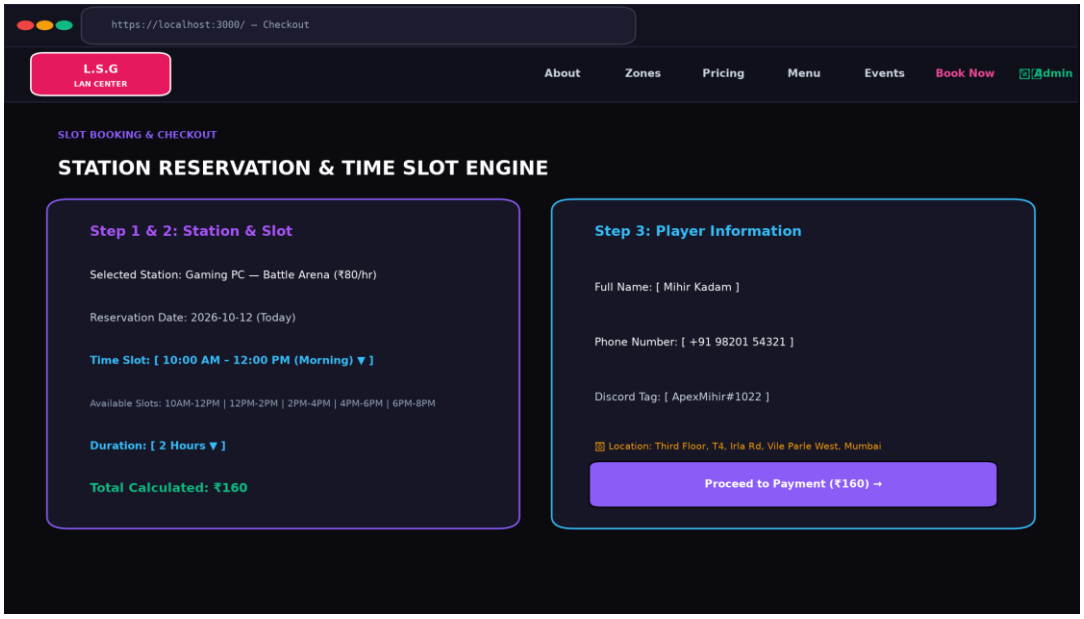



*Figure 4.4 Interactive station reservation and 10 AM – 8 PM time slot picker*

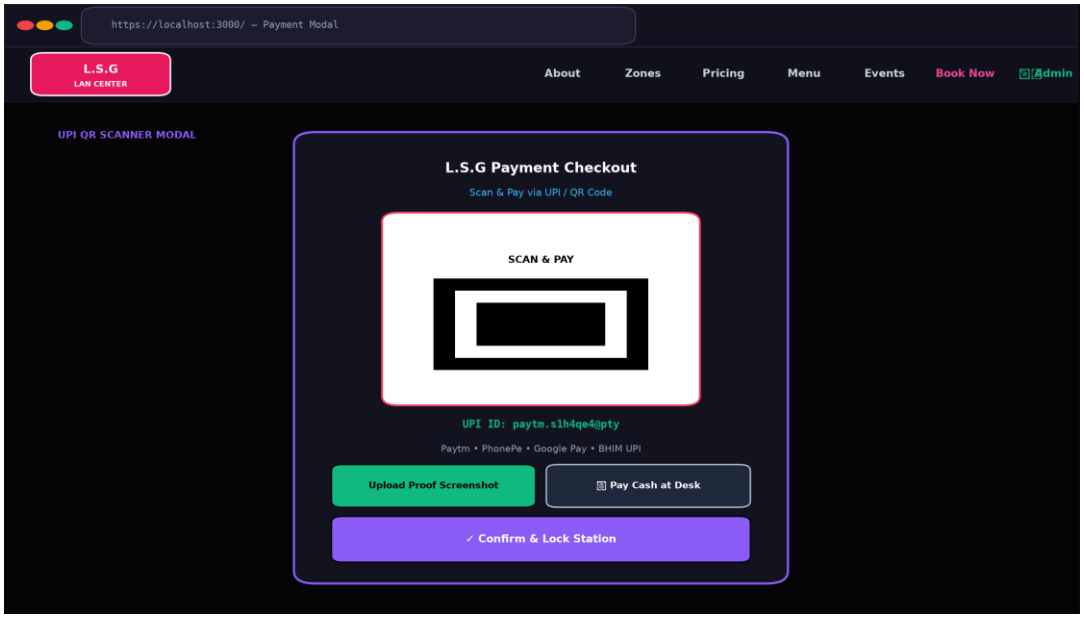



*Figure 4.5 Dual-mode checkout modal featuring UPI QR scanner and cash toggle*

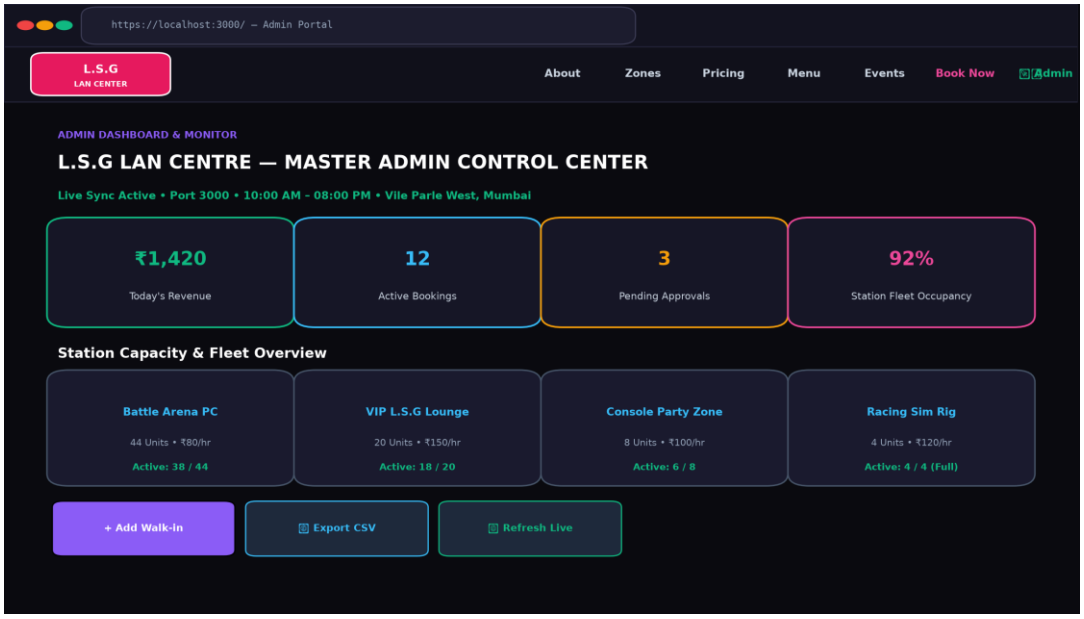



*Figure 4.6 Master administrator dashboard with live station fleet monitor*

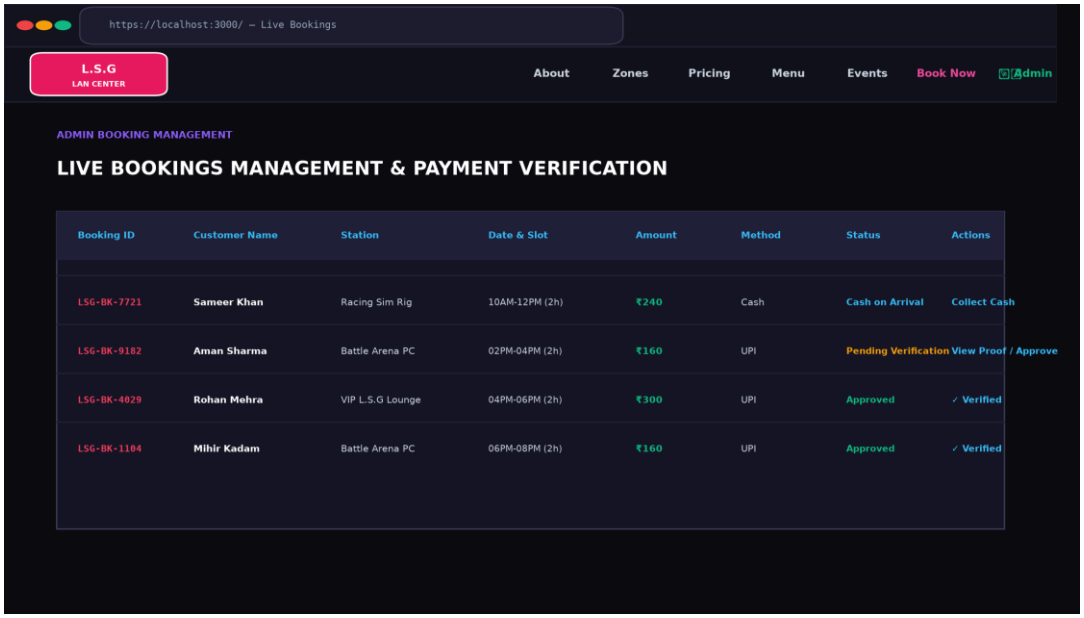



*Figure 4.7 Live bookings verification table with real-time approval workflow*

## 4.4 Coding

This section includes verbatim, authentic source code from the L.S.G LAN CENTRE application implementation. The code illustrates the administrative interface, the client booking portal, and the live Node.js REST and SSE server.

### VERBATIM SOURCE CODE — SERVER.JS (NODE.JS HTTP & SSE ENGINE)

```
// ============================================
// L.S.G LAN CENTRE — Live Web & Admin Server
// ============================================

const http = require('http');
const fs = require('fs');
const path = require('path');

const PORT = 3000;
const DATA_FILE = path.join(__dirname, 'bookings.json');

// MIME types for static assets
const MIME_TYPES = {
  '.html': 'text/html; charset=utf-8',
  '.css': 'text/css; charset=utf-8',
  '.js': 'application/javascript; charset=utf-8',
  '.json': 'application/json; charset=utf-8',
  '.png': 'image/png',
  '.jpg': 'image/jpeg',
  '.jpeg': 'image/jpeg',
  '.webp': 'image/webp',
  '.svg': 'image/svg+xml',
  '.ico': 'image/x-icon'
};

// SSE connected clients
const sseClients = new Set();

function broadcastSSE(event, data) {
  const message = `event: ${event}\ndata: ${JSON.stringify(data)}\n\n`;
```

```
  for (const res of sseClients) {
    try {
      res.write(message);
    } catch (e) {
      sseClients.delete(res);
    }
  }
}

// Read bookings from disk
function readBookings() {
  try {
    if (!fs.existsSync(DATA_FILE)) {
      // Seed initial sample bookings
      const initial = [
        {
          bookingId: 'LSG-BK-7721',
          stationId: 'racing-sim',
          stationName: 'Racing Simulator Rig',
          date: new Date().toISOString().split('T')[0],
          slot: '08:00 PM – 10:00 PM',
          duration: '2 Hour(s)',
          totalAmount: '₹240',
          paymentMethod: 'Cash',
          customer: {
            name: 'Sameer Khan',
            phone: '+91 97722 11002',
            email: 'sameer.f1@gmail.com',
            discord: 'ApexSameer#4040'
          },
          paymentStatus: 'Cash on Arrival',
          proof: {
            fileName: 'Cash Payment at Counter',
            fileSize: 0,
            fileType: 'cash',
            dataUrl: null,
            note: 'Customer will pay physical cash at the front reception desk upon arrival',
            uploadedAt: new Date().toISOString()
          },
          createdAt: new Date().toISOString()
        },
        {
          bookingId: 'LSG-BK-9182',
          stationId: 'battle-arena',
          stationName: 'Gaming PC – Battle Arena',
          date: new Date().toISOString().split('T')[0],
          slot: '06:00 PM – 08:00 PM',
          duration: '2 Hour(s)',
          totalAmount: '₹160',
          paymentMethod: 'UPI',
          customer: {
            name: 'Aman Sharma',
            phone: '+91 98201 54321',
            email: 'aman.gamer@gmail.com',
            discord: 'AmanApex#9921'
          },
          paymentStatus: 'Pending Verification',
          proof: {
            fileName: 'paytm_transaction.png',
            fileSize: 469260,
            fileType: 'image/png',
            dataUrl: 'images/payment-scanner.png',
            uploadedAt: new Date().toISOString()
          },
          createdAt: new Date().toISOString()
        },
        {
          bookingId: 'LSG-BK-4029',
          stationId: 'vip-lounge',
```

```
          stationName: 'VIP Gaming PC – L.S.G Lounge',
          date: new Date().toISOString().split('T')[0],
          slot: '02:00 PM – 04:00 PM',
          duration: '2 Hour(s)',
          totalAmount: '₹300',
          paymentMethod: 'UPI',
          customer: {
            name: 'Rohan Mehra',
            phone: '+91 98112 34567',
            email: 'rohan.fps@yahoo.com',
            discord: 'RohanViper#1022'
          },
          paymentStatus: 'Approved',
          proof: {
            fileName: 'upi_screenshot_rohan.jpg',
            fileSize: 312000,
            fileType: 'image/jpeg',
            dataUrl: 'images/payment-scanner.png',
            uploadedAt: new Date(Date.now() - 3600000).toISOString()
          },
          createdAt: new Date(Date.now() - 3600000).toISOString()
        }
      ];
      fs.writeFileSync(DATA_FILE, JSON.stringify(initial, null, 2), 'utf-8');
      return initial;
    }
    const raw = fs.readFileSync(DATA_FILE, 'utf-8');
    return JSON.parse(raw);
  } catch (err) {
    console.error('Error reading bookings file:', err);
    return [];
  }
}

// Write bookings to disk
function writeBookings(data) {
  try {
    fs.writeFileSync(DATA_FILE, JSON.stringify(data, null, 2), 'utf-8');
    return true;
  } catch (err) {
    console.error('Error writing bookings file:', err);
    return false;
  }
}

const server = http.createServer((req, res) => {
  // CORS Headers
  res.setHeader('Access-Control-Allow-Origin', '*');
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

  if (req.method === 'OPTIONS') {
    res.writeHead(204);
    res.end();
    return;
  }

  const parsedUrl = new URL(req.url, `http://${req.headers.host || 'localhost'}`);
  const pathname = parsedUrl.pathname;

  // ---------- API: Live SSE Stream ----------
  if (pathname === '/api/live-stream') {
    res.writeHead(200, {
      'Content-Type': 'text/event-stream',
      'Cache-Control': 'no-cache',
      'Connection': 'keep-alive'
    });
    res.write('retry: 3000\n\n');
```

```
    sseClients.add(res);

    req.on('close', () => {
      sseClients.delete(res);
    });
    return;
  }

  // ---------- API: Get All Bookings ----------
  if (req.method === 'GET' && (pathname === '/api/bookings' || pathname === '/api/payment-
proof')) {
    const list = readBookings();
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify(list));
    return;
  }

  // ---------- API: Create New Booking ----------
  if (req.method === 'POST' && (pathname === '/api/bookings' || pathname === '/api/payment-
proof')) {
    let body = '';
    req.on('data', chunk => {
      body += chunk;
      // Protect against gigantic payloads (> 20MB)
      if (body.length > 20 * 1024 * 1024) {
        res.writeHead(413, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ error: 'Payload too large' }));
        req.destroy();
      }
    });

    req.on('end', () => {
      try {
        const newBooking = JSON.parse(body);
        if (!newBooking.bookingId) {
          newBooking.bookingId = `LSG-BK-${Math.floor(1000 + Math.random() * 9000)}`;
        }
        if (!newBooking.createdAt) {
          newBooking.createdAt = new Date().toISOString();
        }
        if (!newBooking.paymentStatus) {
          newBooking.paymentStatus = 'Pending Verification';
        }

        const bookings = readBookings();
        // Insert at beginning
        bookings.unshift(newBooking);
        writeBookings(bookings);

        // Notify all live tabs and clients via SSE
        broadcastSSE('booking_created', newBooking);

        res.writeHead(201, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ success: true, booking: newBooking }));
      } catch (err) {
        res.writeHead(400, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ error: 'Invalid JSON payload' }));
      }
    });
    return;
  }

  // ---------- API: Update Booking Status ----------
  if (req.method === 'PUT' && pathname.startsWith('/api/bookings/')) {
    const bookingId = decodeURIComponent(pathname.replace('/api/bookings/', ''));
    let body = '';
    req.on('data', chunk => { body += chunk; });
```

```
    req.on('end', () => {
      try {
        const updates = JSON.parse(body);
        const bookings = readBookings();
        const target = bookings.find(b => b.bookingId === bookingId);

        if (!target) {
          res.writeHead(404, { 'Content-Type': 'application/json' });
          res.end(JSON.stringify({ error: 'Booking not found' }));
          return;
        }

        Object.assign(target, updates);
        writeBookings(bookings);

        // Notify live listeners
        broadcastSSE('booking_updated', target);

        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ success: true, booking: target }));
      } catch (err) {
        res.writeHead(400, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ error: 'Invalid update payload' }));
      }
    });
    return;
  }

  // ---------- API: Delete Booking ----------
  if (req.method === 'DELETE' && pathname.startsWith('/api/bookings/')) {
    const bookingId = decodeURIComponent(pathname.replace('/api/bookings/', ''));
    const bookings = readBookings();
    const filtered = bookings.filter(b => b.bookingId !== bookingId);

    if (filtered.length === bookings.length) {
      res.writeHead(404, { 'Content-Type': 'application/json' });
      res.end(JSON.stringify({ error: 'Booking not found' }));
      return;
    }

    writeBookings(filtered);
    broadcastSSE('booking_deleted', { bookingId });

    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ success: true, bookingId }));
    return;
  }

  // ---------- API: Clear All Bookings ----------
  if (req.method === 'DELETE' && pathname === '/api/bookings') {
    writeBookings([]);
    broadcastSSE('bookings_cleared', {});
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ success: true }));
    return;
  }

  // ---------- Static File Serving ----------
  let relativePath = pathname === '/' ? 'index.html' : pathname;
  // Strip leading slash
  if (relativePath.startsWith('/')) {
    relativePath = relativePath.slice(1);
  }
  // If request has no extension, check if an html file exists (e.g. /admin -> admin.html)
  if (!path.extname(relativePath)) {
    if (fs.existsSync(path.join(__dirname, relativePath + '.html'))) {
      relativePath += '.html';
```

```
    }
  }

  const filePath = path.join(__dirname, relativePath);

  // Security check: prevent directory traversal
  if (!filePath.startsWith(__dirname)) {
    res.writeHead(403, { 'Content-Type': 'text/plain' });
    res.end('Access Denied');
    return;
  }

  fs.stat(filePath, (err, stats) => {
    if (err || !stats.isFile()) {
      res.writeHead(404, { 'Content-Type': 'text/plain' });
      res.end('404 Not Found');
      return;
    }

    const ext = path.extname(filePath).toLowerCase();
    const contentType = MIME_TYPES[ext] || 'application/octet-stream';

    res.writeHead(200, {
      'Content-Type': contentType,
      'Content-Length': stats.size,
      'Cache-Control': 'no-cache, no-store, must-revalidate',
      'Pragma': 'no-cache',
      'Expires': '0'
    });

    const stream = fs.createReadStream(filePath);
    stream.pipe(res);
  });
});

server.listen(PORT, () => {
  console.log(`L.S.G Server running live at http://localhost:${PORT}`);
  console.log(`Main Webpage:  http://localhost:${PORT}/`);
  console.log(`Admin Portal:  http://localhost:${PORT}/admin.html`);
});
```

## VERBATIM SOURCE CODE — ADMIN.HTML (ADMIN PORTAL & VERIFICATION UI)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>L.S.G LAN CENTRE — Admin Portal &amp;amp; Verification</title>
  <link rel="stylesheet" href="admin.css?v=5.0">
  <link rel="icon" type="image/png" href="images/lsg-logo.png">
</head>
<body>

  <div class="admin-layout">
    <!-- Navbar -->
    <header class="admin-navbar">
      <div class="admin-navbar-content">
        <div class="brand-section">
          <a href="admin.html" class="brand-badge">
            <img src="images/lsg-logo.png" alt="L.S.G LAN CENTRE Logo" class="brand-logo-img">
            <div>
              <div class="brand-title">L.S.G LAN CENTRE</div>
              <span class="portal-tag">Admin Control Center</span>
            </div>
          </a>
```

```
          <div class="system-status">
            <span class="pulse-dot"></span>
            <span>Live Sync Active</span>
          </div>
        </div>

        <div class="nav-actions">
          <button type="button" class="btn-admin btn-admin-secondary" id="refreshBtn"
title="Refresh bookings data">
            <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5"><polyline points="23 4 23 10 17 10"/><polyline points="1 20 1 14 7 14"/><path
d="M3.51 9a9 9 0 0 1 14.85-3.36L23 10M1 14l4.64 4.36A9 9 0 0 0 20.49 15"/></svg>
            <span>Refresh</span>
          </button>
          <button type="button" class="btn-admin btn-admin-secondary" id="exportCsvBtn"
title="Download Excel/CSV report">
            <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4"/><polyline points="7 10
12 15 17 10"/><line x1="12" y1="15" x2="12" y2="3"/></svg>
            <span>Export CSV</span>
          </button>
          <button type="button" class="btn-admin btn-admin-primary" id="addWalkinBtn">
            <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5"><line x1="12" y1="5" x2="12" y2="19"/><line x1="5" y1="12" x2="19"
y2="12"/></svg>
            <span>Add Walk-in</span>
          </button>
          <a href="index.html" class="btn-admin btn-admin-secondary" style="border-color: var(--
accent-secondary);" title="Go to customer-facing website">
            <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5"><path d="M18 13v6a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V8a2 2 0 0 1 2-2h6"/><polyline
points="15 3 21 3 21 9"/><line x1="10" y1="14" x2="21" y2="3"/></svg>
            <span>Customer Site</span>
          </a>
        </div>
      </div>
    </header>

    <!-- Main Content -->
    <main class="admin-main">

      <!-- Stats KPI Row -->
      <section class="stats-grid">
        <!-- Pending Verification Card -->
        <div class="stat-card">
          <div class="stat-info">
            <span>Pending Verification</span>
            <div class="stat-value pending-color" id="statPendingCount">0</div>
          </div>
          <div class="stat-icon pending">⏳</div>
        </div>

        <!-- Verified / Approved Card -->
        <div class="stat-card">
          <div class="stat-info">
            <span>Verified Bookings</span>
            <div class="stat-value verified-color" id="statVerifiedCount">0</div>
          </div>
          <div class="stat-icon verified">☑</div>
        </div>

        <!-- Total Revenue Card -->
        <div class="stat-card">
          <div class="stat-info">
            <span>Confirmed Revenue</span>
            <div class="stat-value revenue-color" id="statRevenueTotal">₹0</div>
          </div>
          <div class="stat-icon revenue">💰</div>
```

```
        </div>

        <!-- Total Bookings Card -->
        <div class="stat-card">
          <div class="stat-info">
            <span>Total Reservations</span>
            <div class="stat-value" id="statTotalBookings">0</div>
          </div>
          <div class="stat-icon stations">🎮</div>
        </div>
      </section>

      <!-- Live Station Rig Monitor -->
      <section class="station-monitor-card">
        <div class="section-head">
          <h3>
            <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5"><rect x="2" y="3" width="20" height="14" rx="2"/><line x1="8" y1="21" x2="16"
y2="21"/><line x1="12" y1="17" x2="12" y2="21"/></svg>
            Station Capacity &amp;amp; Fleet Overview
          </h3>
          <span style="font-size: 0.8rem; color: var(--accent-secondary); font-weight:
600;">10:00 AM – 08:00 PM • Vile Parle West, Mumbai</span>
        </div>

        <div class="rig-cards-row">
          <div class="rig-status-pill">
            <div class="rig-icon-box">💻</div>
            <div>
              <div class="rig-title">Battle Arena PC</div>
              <div class="rig-count">44 Stations • ₹80/hr</div>
            </div>
          </div>
          <div class="rig-status-pill">
            <div class="rig-icon-box">👑</div>
            <div>
              <div class="rig-title">VIP L.S.G Lounge</div>
              <div class="rig-count">20 Stations • ₹150/hr</div>
            </div>
          </div>
          <div class="rig-status-pill">
            <div class="rig-icon-box">🎮</div>
            <div>
              <div class="rig-title">PS5 Party Zone</div>
              <div class="rig-count">4 Consoles • ₹100/hr</div>
            </div>
          </div>
          <div class="rig-status-pill">
            <div class="rig-icon-box">🏎️</div>
            <div>
              <div class="rig-title">Racing Sim Rigs</div>
              <div class="rig-count">2 Pods • ₹120/hr</div>
            </div>
          </div>
        </div>
      </section>

      <!-- Reservations Management Panel -->
      <section class="panel-card">
        <div class="table-toolbar">
          <!-- Filter Tabs -->
          <div class="filter-tabs">
            <button type="button" class="filter-tab active" data-filter="all">All</button>
            <button type="button" class="filter-tab" data-filter="pending">Pending
Verification</button>
            <button type="button" class="filter-tab" data-filter="cash">💵 Cash on
Arrival</button>
```

```
            <button type="button" class="filter-tab" data-filter="approved">Verified /
Approved</button>
            <button type="button" class="filter-tab" data-filter="rejected">Rejected</button>
          </div>

          <!-- Search & Clear -->
          <div class="toolbar-controls">
            <div class="search-box">
              <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2"><circle cx="11" cy="11" r="8"/><line x1="21" y1="21" x2="16.65"
y2="16.65"/></svg>
              <input type="text" id="adminSearchInput" placeholder="Search by name, ID,
phone...">
            </div>
            <button type="button" class="btn-admin btn-admin-danger btn-admin-sm"
id="clearDataBtn" title="Clear all reservation records">
              <span>Clear History</span>
            </button>
          </div>
        </div>

        <!-- Table Container -->
        <div class="table-responsive">
          <table class="reservations-table">
            <thead>
              <tr>
                <th>Booking ID</th>
                <th>Customer</th>
                <th>Gaming Station</th>
                <th>Date &amp;amp; Slot</th>
                <th>Amount</th>
                <th>Payment Mode &amp;amp; Proof</th>
                <th>Verification Status</th>
                <th>Actions</th>
              </tr>
            </thead>
            <tbody id="reservationsTableBody">
              <!-- Dynamically populated via admin.js -->
            </tbody>
          </table>

          <div class="table-empty-state" id="emptyState" style="display: none;">
            <svg width="48" height="48" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="1.5"><rect x="3" y="3" width="18" height="18" rx="2" ry="2"/><line x1="9" y1="9"
x2="15" y2="15"/><line x1="15" y1="9" x2="9" y2="15"/></svg>
            <h4>No Reservations Found</h4>
            <p>No bookings match the selected filter criteria. New customer submissions will
appear here automatically.</p>
          </div>
        </div>
      </section>

    </main>
  </div>

  <!-- ====== FULL PROOF VIEWER MODAL ====== -->
  <div class="admin-modal-overlay" id="proofModal" role="dialog" aria-modal="true">
    <div class="admin-modal-box">
      <div class="admin-modal-header">
        <h3 id="modalProofTitle">Payment Verification</h3>
        <button type="button" class="btn-action-icon" id="closeProofModalBtn">&times;</button>
      </div>

      <div class="admin-modal-body">
        <div class="proof-full-preview" id="proofImgWrap">
          <img id="fullProofImg" src="images/payment-scanner.png?v=2" alt="Payment Proof
Screenshot">
        </div>
```

```
        <!-- Cash notice if cash payment -->
        <div class="cash-desk-notice" id="modalProofCashNotice" style="display: none;">
          <div class="cash-notice-icon">💵</div>
          <div class="cash-notice-content">
            <h4>Cash on Arrival (Pay at Reception Desk)</h4>
            <p>This customer chose to pay in physical cash upon arrival. No UPI screenshot is
required.</p>
            <div class="cash-collect-highlight">
              <span>Collect <strong id="modalProofCashAmount" style="color: #34d399; font-size:
1.1rem;">₹80</strong> in cash at the counter before approving.</span>
            </div>
          </div>
        </div>

        <div class="proof-meta-grid">
          <div class="meta-item">
            <span class="meta-label">Booking Reference:</span>
            <span class="meta-val booking-code" id="modalProofBookingId">LSG-BK-9182</span>
          </div>
          <div class="meta-item">
            <span class="meta-label">Customer Name:</span>
            <span class="meta-val" id="modalProofCustomer">Aman Sharma</span>
          </div>
          <div class="meta-item">
            <span class="meta-label">Payment Method:</span>
            <span class="meta-val" id="modalProofMethod" style="color: var(--accent-cyan); font-
weight: 700;">UPI</span>
          </div>
          <div class="meta-item">
            <span class="meta-label">Payable Amount:</span>
            <span class="meta-val amount-cell" id="modalProofAmount">₹160</span>
          </div>
          <div class="meta-item">
            <span class="meta-label">Station:</span>
            <span class="meta-val" id="modalProofStation">Gaming PC — Battle Arena</span>
          </div>
          <div class="meta-item">
            <span class="meta-label">Date &amp;amp; Slot:</span>
            <span class="meta-val" id="modalProofDate">Today • 06:00 PM – 08:00 PM</span>
          </div>
          <div class="meta-item">
            <span class="meta-label">Current Status:</span>
            <span class="meta-val" id="modalProofStatus">Pending Verification</span>
          </div>
        </div>
      </div>

      <div class="admin-modal-footer">
        <button type="button" class="btn-admin btn-admin-danger" id="modalRejectBtn">
          <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18"
y2="18"/></svg>
          Reject Payment
        </button>
        <button type="button" class="btn-admin btn-admin-primary" id="modalApproveBtn">
          <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5"><polyline points="20 6 9 17 4 12"/></svg>
          Verify &amp;amp; Approve
        </button>
      </div>
    </div>
  </div>

  <!-- ====== ADD WALK-IN MODAL ====== -->
  <div class="admin-modal-overlay" id="walkinModal" role="dialog" aria-modal="true">
    <div class="admin-modal-box" style="max-width: 480px;">
      <div class="admin-modal-header">
```

```
        <h3>Add Walk-in Counter Booking</h3>
        <button type="button" class="btn-action-icon" id="closeWalkinModalBtn">&times;</button>
      </div>

      <form id="walkinForm">
        <div class="admin-modal-body">
          <div style="display: flex; flex-direction: column; gap: 0.5rem;">
            <label style="font-size: 0.8rem; font-weight: 600; color: var(--text-muted); text-
transform: uppercase;">Customer Name *</label>
            <input type="text" id="walkinName" required placeholder="Player Name"
style="padding: 0.75rem; background: rgba(255,255,255,0.05); border: 1px solid var(--border-
subtle); border-radius: 10px; color: #fff; font-family: inherit;">
          </div>

          <div style="display: flex; flex-direction: column; gap: 0.5rem;">
            <label style="font-size: 0.8rem; font-weight: 600; color: var(--text-muted); text-
transform: uppercase;">Phone Number *</label>
            <input type="tel" id="walkinPhone" required placeholder="+91 98765 43210"
style="padding: 0.75rem; background: rgba(255,255,255,0.05); border: 1px solid var(--border-
subtle); border-radius: 10px; color: #fff; font-family: inherit;">
          </div>

          <div style="display: flex; flex-direction: column; gap: 0.5rem;">
            <label style="font-size: 0.8rem; font-weight: 600; color: var(--text-muted); text-
transform: uppercase;">Select Station</label>
            <select id="walkinStation" style="padding: 0.75rem; background: #1a1a28; border: 1px
solid var(--border-subtle); border-radius: 10px; color: #fff; font-family: inherit;">
              <option value="battle-arena" data-rate="80">Gaming PC — Battle Arena
(₹80/hr)</option>
              <option value="vip-lounge" data-rate="150">VIP Gaming PC — L.S.G Lounge
(₹150/hr)</option>
              <option value="console-zone" data-rate="100">PS5 Console — Party Zone
(₹100/hr)</option>
              <option value="racing-sim" data-rate="120">Racing Simulator Rig (₹120/hr)</option>
            </select>
          </div>

          <div style="display: flex; flex-direction: column; gap: 0.5rem;">
            <label style="font-size: 0.8rem; font-weight: 600; color: var(--text-muted); text-
transform: uppercase;">Duration</label>
            <select id="walkinDuration" style="padding: 0.75rem; background: #1a1a28; border:
1px solid var(--border-subtle); border-radius: 10px; color: #fff; font-family: inherit;">
              <option value="1">1 Hour</option>
              <option value="2" selected>2 Hours</option>
              <option value="3">3 Hours</option>
              <option value="4">4 Hours</option>
              <option value="8">8 Hours (Full Day)</option>
            </select>
          </div>

          <div style="display: flex; flex-direction: column; gap: 0.5rem;">
            <label style="font-size: 0.8rem; font-weight: 600; color: var(--text-muted); text-
transform: uppercase;">Payment Method</label>
            <select id="walkinPaymentMethod" style="padding: 0.75rem; background: #1a1a28;
border: 1px solid var(--border-subtle); border-radius: 10px; color: #fff; font-family:
inherit;">
              <option value="Cash" selected>💵 Cash at Counter (Paid)</option>
              <option value="UPI">📱 UPI / QR Code (Paid)</option>
            </select>
          </div>
        </div>

        <div class="admin-modal-footer">
          <button type="button" class="btn-admin btn-admin-secondary"
id="cancelWalkinBtn">Cancel</button>
          <button type="submit" class="btn-admin btn-admin-primary">Create Booking</button>
        </div>
      </form>
```

```
    </div>
  </div>

  <!-- Toast Notification -->
  <div class="admin-toast" id="adminToast">
    <svg width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2.5"><polyline points="20 6 9 17 4 12"/></svg>
    <span id="toastMessage">Action completed successfully</span>
  </div>

  <script src="admin.js?v=5.0"></script>
</body>
</html>
```

## VERBATIM SOURCE CODE — ADMIN.JS (ADMIN DASHBOARD CONTROLLER & SSE LISTENER)

```
// ==========================================
// L.S.G LAN CENTRE — Admin Portal Logic (Live Sync)
// ==========================================

document.addEventListener('DOMContentLoaded', () => {

  const STORAGE_KEY = 'lsg_reservations';

  // State
  let reservations = [];
  let currentFilter = 'all';
  let searchQuery = '';
  let activeModalBooking = null;

  // DOM Elements
  const statPendingCount = document.getElementById('statPendingCount');
  const statVerifiedCount = document.getElementById('statVerifiedCount');
  const statRevenueTotal = document.getElementById('statRevenueTotal');
  const statTotalBookings = document.getElementById('statTotalBookings');
  const tableBody = document.getElementById('reservationsTableBody');
  const emptyState = document.getElementById('emptyState');
  const searchInput = document.getElementById('adminSearchInput');
  const filterTabs = document.querySelectorAll('.filter-tab');
  const refreshBtn = document.getElementById('refreshBtn');
  const exportCsvBtn = document.getElementById('exportCsvBtn');
  const clearDataBtn = document.getElementById('clearDataBtn');
  const addWalkinBtn = document.getElementById('addWalkinBtn');

  // Proof Modal Elements
  const proofModal = document.getElementById('proofModal');
  const closeProofModalBtn = document.getElementById('closeProofModalBtn');
  const fullProofImg = document.getElementById('fullProofImg');
  const modalProofBookingId = document.getElementById('modalProofBookingId');
  const modalProofCustomer = document.getElementById('modalProofCustomer');
  const modalProofAmount = document.getElementById('modalProofAmount');
  const modalProofStation = document.getElementById('modalProofStation');
  const modalProofDate = document.getElementById('modalProofDate');
  const modalProofStatus = document.getElementById('modalProofStatus');
  const modalApproveBtn = document.getElementById('modalApproveBtn');
  const modalRejectBtn = document.getElementById('modalRejectBtn');

  // Walk-in Modal Elements
  const walkinModal = document.getElementById('walkinModal');
  const closeWalkinModalBtn = document.getElementById('closeWalkinModalBtn');
  const cancelWalkinBtn = document.getElementById('cancelWalkinBtn');
  const walkinForm = document.getElementById('walkinForm');

  // Toast
  const adminToast = document.getElementById('adminToast');
```

```
const toastMessage = document.getElementById('toastMessage');

// Audio Chime (Web Audio API)
function playAlertChime() {
  try {
    const ctx = new (window.AudioContext || window.webkitAudioContext)();
    const osc = ctx.createOscillator();
    const gain = ctx.createGain();
    osc.type = 'triangle';
    osc.frequency.setValueAtTime(440, ctx.currentTime);
    osc.frequency.setValueAtTime(880, ctx.currentTime + 0.1);
    gain.gain.setValueAtTime(0.15, ctx.currentTime);
    gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.4);
    osc.connect(gain);
    gain.connect(ctx.destination);
    osc.start();
    osc.stop(ctx.currentTime + 0.4);
  } catch (e) {}
}

// Load from Server API and LocalStorage
async function loadReservations() {
  // 1. Try fetching from live server API
  try {
    const res = await fetch('/api/bookings');
    if (res.ok) {
      const serverData = await res.json();
      if (Array.isArray(serverData) && serverData.length > 0) {
        reservations = serverData;
        localStorage.setItem(STORAGE_KEY, JSON.stringify(reservations));
        updateKPIs();
        renderTable();
        return;
      }
    }
  } catch (err) {
    // Backend not running, fallback to localStorage
  }

  // 2. Fallback to LocalStorage
  try {
    const data = localStorage.getItem(STORAGE_KEY);
    if (data) {
      reservations = JSON.parse(data);
    } else {
      // Seed default sample bookings if completely empty
      reservations = [
        {
          bookingId: 'LSG-BK-7721',
          stationId: 'racing-sim',
          stationName: 'Racing Simulator Rig',
          date: new Date().toISOString().split('T')[0],
          slot: '08:00 PM – 10:00 PM',
          duration: '2 Hour(s)',
          totalAmount: '₹240',
          paymentMethod: 'Cash',
          customer: {
            name: 'Sameer Khan',
            phone: '+91 97722 11002',
            email: 'sameer.f1@gmail.com',
            discord: 'ApexSameer#4040'
          },
          paymentStatus: 'Cash on Arrival',
          proof: {
            fileName: 'Cash Payment at Counter',
            fileSize: 0,
            fileType: 'cash',
            dataUrl: null,
            note: 'Customer will pay physical cash at the front reception desk upon arrival',
```

```
            uploadedAt: new Date().toISOString()
          },
          createdAt: new Date().toISOString()
        },
        {
          bookingId: 'LSG-BK-9182',
          stationId: 'battle-arena',
          stationName: 'Gaming PC — Battle Arena',
          date: new Date().toISOString().split('T')[0],
          slot: '06:00 PM – 08:00 PM',
          duration: '2 Hour(s)',
          totalAmount: '₹160',
          paymentMethod: 'UPI',
          customer: {
            name: 'Aman Sharma',
            phone: '+91 98201 54321',
            email: 'aman.gamer@gmail.com',
            discord: 'AmanApex#9921'
          },
          paymentStatus: 'Pending Verification',
          proof: {
            fileName: 'paytm_transaction.png',
            fileSize: 469260,
            fileType: 'image/png',
            dataUrl: 'images/payment-scanner.png',
            uploadedAt: new Date().toISOString()
          },
          createdAt: new Date().toISOString()
        },
        {
          bookingId: 'LSG-BK-4029',
          stationId: 'vip-lounge',
          stationName: 'VIP Gaming PC — L.S.G Lounge',
          date: new Date().toISOString().split('T')[0],
          slot: '02:00 PM – 04:00 PM',
          duration: '2 Hour(s)',
          totalAmount: '₹300',
          paymentMethod: 'UPI',
          customer: {
            name: 'Rohan Mehra',
            phone: '+91 98112 34567',
            email: 'rohan.fps@yahoo.com',
            discord: 'RohanViper#1022'
          },
          paymentStatus: 'Approved',
          proof: {
            fileName: 'upi_screenshot_rohan.jpg',
            fileSize: 312000,
            fileType: 'image/jpeg',
            dataUrl: 'images/payment-scanner.png',
            uploadedAt: new Date(Date.now() - 3600000).toISOString()
          },
          createdAt: new Date(Date.now() - 3600000).toISOString()
        }
      ];
      saveReservations();
    }
  } catch (e) {
    console.error('Error reading reservations:', e);
    reservations = [];
  }

  updateKPIs();
  renderTable();
}

function saveReservations() {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(reservations));
```

```
    } catch (e) {
      console.error('Error saving reservations:', e);
    }
  }

  // Update KPI Stats Cards
  function updateKPIs() {
    const total = reservations.length;
    let pending = 0;
    let verified = 0;
    let revenue = 0;

    reservations.forEach(r => {
      const status = (r.paymentStatus || '').toLowerCase();
      if (status.includes('pending')) {
        pending++;
      } else if (status.includes('approved') || status.includes('verified')) {
        verified++;
        const amtStr = String(r.totalAmount || '0').replace(/[^0-9]/g, '');
        revenue += parseInt(amtStr, 10) || 0;
      }
    });

    if (statPendingCount) statPendingCount.textContent = pending;
    if (statVerifiedCount) statVerifiedCount.textContent = verified;
    if (statRevenueTotal) statRevenueTotal.textContent = `₹${revenue.toLocaleString('en-IN')}`;
    if (statTotalBookings) statTotalBookings.textContent = total;
  }

  // Filter & Search Logic
  function getFilteredReservations() {
    return reservations.filter(r => {
      // Status filter
      const status = (r.paymentStatus || '').toLowerCase();
      const isCash = (r.paymentMethod || '').toLowerCase() === 'cash' ||
status.includes('cash');

      if (currentFilter === 'cash' && !isCash) return false;
      if (currentFilter === 'pending' && !status.includes('pending') && !isCash) return false;
      if (currentFilter === 'approved' && !status.includes('approved') &&
!status.includes('verified')) return false;
      if (currentFilter === 'rejected' && !status.includes('rejected')) return false;

      // Search filter
      if (searchQuery) {
        const q = searchQuery.toLowerCase();
        const idMatch = (r.bookingId || '').toLowerCase().includes(q);
        const nameMatch = (r.customer && r.customer.name || '').toLowerCase().includes(q);
        const phoneMatch = (r.customer && r.customer.phone || '').toLowerCase().includes(q);
        const emailMatch = (r.customer && r.customer.email || '').toLowerCase().includes(q);
        const stationMatch = (r.stationName || '').toLowerCase().includes(q);
        const methodMatch = (r.paymentMethod || '').toLowerCase().includes(q);
        if (!idMatch && !nameMatch && !phoneMatch && !emailMatch && !stationMatch &&
!methodMatch) {
          return false;
        }
      }

      return true;
    });
  }

  // Render Table
  function renderTable() {
    const list = getFilteredReservations();

    if (!tableBody) return;
```

```
    tableBody.innerHTML = '';

    if (list.length === 0) {
      if (emptyState) emptyState.style.display = 'block';
      return;
    }

    if (emptyState) emptyState.style.display = 'none';

    list.forEach(r => {
      const tr = document.createElement('tr');
      tr.id = `row-${r.bookingId}`;

      const isCash = (r.paymentMethod || '').toLowerCase() === 'cash' || (r.paymentStatus ||
'').toLowerCase().includes('cash');
      const isApproved = (r.paymentStatus || '').toLowerCase().includes('approved') ||
(r.paymentStatus || '').toLowerCase().includes('verified');
      const isRejected = (r.paymentStatus || '').toLowerCase().includes('rejected');
      const isPending = !isApproved && !isRejected;

      let statusBadgeClass = 'pending';
      let statusText = 'Pending Verification';
      if (isApproved) {
        statusBadgeClass = 'approved';
        statusText = isCash ? 'Paid (Cash Verified)' : 'Verified / Approved';
      } else if (isRejected) {
        statusBadgeClass = 'rejected';
        statusText = 'Rejected';
      } else if (isCash) {
        statusBadgeClass = 'cash';
        statusText = 'Cash on Arrival';
      }

      // Proof / Payment Mode Preview HTML
      let proofHtml = '<span class="no-proof">None</span>';
      if (isCash) {
        proofHtml = `
          <button type="button" class="badge-cash" data-booking-id="${r.bookingId}" title="Cash
on arrival — Click to view desk payment details">
            <span class="cash-dot"></span>
            <span>💵 Cash at Desk</span>
          </button>
        `;
      } else if (r.proof) {
        const imgSrc = (r.proof && r.proof.dataUrl) ? r.proof.dataUrl : 'images/payment-
scanner.png';
        proofHtml = `
          <button type="button" class="proof-thumbnail-btn" data-booking-id="${r.bookingId}"
title="Click to view full screenshot">
            <img src="${imgSrc}" class="proof-thumb" alt="Proof thumbnail">
            <span style="font-size: 0.75rem; color: var(--accent-cyan); font-weight:
600;">View</span>
          </button>
        `;
      }

      tr.innerHTML = `
        <td><span class="booking-code">${r.bookingId || 'N/A'}</span></td>
        <td>
          <div class="customer-cell">
            <span class="customer-name">${escapeHtml(r.customer && r.customer.name ||
'Anonymous')}</span>
            <span class="customer-contact">${escapeHtml(r.customer && r.customer.phone ||
'')}</span>
            <span class="customer-contact">${escapeHtml(r.customer && r.customer.email ||
'')}</span>
          </div>
        </td>
```

```
        <td>
          <div class="station-cell">
            <span>${getStationIcon(r.stationId)}</span>
            <span>${escapeHtml(r.stationName || 'Gaming Rig')}</span>
          </div>
        </td>
        <td>
          <div style="display: flex; flex-direction: column;">
            <span style="font-weight: 600;">${r.date || 'Today'}</span>
            <span style="font-size: 0.75rem; color: var(--text-muted);">${r.slot || ''}
(${r.duration || '1 Hr'})</span>
          </div>
        </td>
        <td><span class="amount-cell">${r.totalAmount || '₹0'}</span></td>
        <td>${proofHtml}</td>
        <td><span class="status-badge ${statusBadgeClass}">${statusText}</span></td>
        <td>
          <div class="action-buttons">
            <button type="button" class="btn-action-icon approve" data-action="approve" data-
id="${r.bookingId}" title="${isCash ? 'Collect Cash & Confirm' : 'Verify & Approve Payment'}">
              <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5"><polyline points="20 6 9 17 4 12"/></svg>
            </button>
            <button type="button" class="btn-action-icon reject" data-action="reject" data-
id="${r.bookingId}" title="Reject Payment">
              <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2.5"><line x1="18" y1="6" x2="6" y2="18"/><line x1="6" y1="6" x2="18"
y2="18"/></svg>
            </button>
            <button type="button" class="btn-action-icon delete" data-action="delete" data-
id="${r.bookingId}" title="Delete Record">
              <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2"><polyline points="3 6 5 6 21 6"/><path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-
2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"/></svg>
            </button>
          </div>
        </td>
      `;

      tableBody.appendChild(tr);
    });
  }

  function getStationIcon(stationId) {
    if (stationId === 'vip-lounge') return '👑';
    if (stationId === 'console-zone') return '🎮';
    if (stationId === 'racing-sim') return '🏎️';
    return '🖥️';
  }

  function escapeHtml(str) {
    if (!str) return '';
    return str.replace(/[&<>"']/g, m => ({
      '&': '&amp;',
      '<': '&lt;',
      '>': '&gt;',
      '"': '&quot;',
      "'": '&#39;'
    }[m]));
  }

  // Toast Notification
  function showToast(msg) {
    if (!adminToast || !toastMessage) return;
    toastMessage.textContent = msg;
    adminToast.classList.add('show');
    setTimeout(() => {
      adminToast.classList.remove('show');
```

```
    }, 3200);
  }

  // Update Status Action (Approve / Reject)
  async function updateBookingStatus(bookingId, newStatus) {
    const booking = reservations.find(r => r.bookingId === bookingId);
    if (!booking) return;

    booking.paymentStatus = newStatus;
    saveReservations();
    updateKPIs();
    renderTable();

    // Broadcast across tabs
    try {
      const channel = new BroadcastChannel('lsg_live_sync');
      channel.postMessage({ type: 'BOOKING_UPDATED', booking });
    } catch (e) {}

    // Send PUT request to server API
    try {
      await fetch(`/api/bookings/${encodeURIComponent(bookingId)}`, {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ paymentStatus: newStatus })
      });
    } catch (e) {}

    if (newStatus === 'Approved') {
      showToast(`Booking ${bookingId} verified and approved!`);
    } else if (newStatus === 'Rejected') {
      showToast(`Booking ${bookingId} marked as rejected.`);
    }
  }

  // Delete Action
  async function deleteBooking(bookingId) {
    if (!confirm(`Are you sure you want to delete booking ${bookingId}?`)) return;
    reservations = reservations.filter(r => r.bookingId !== bookingId);
    saveReservations();
    updateKPIs();
    renderTable();

    // Broadcast across tabs
    try {
      const channel = new BroadcastChannel('lsg_live_sync');
      channel.postMessage({ type: 'BOOKING_DELETED', bookingId });
    } catch (e) {}

    // Send DELETE to server API
    try {
      await fetch(`/api/bookings/${encodeURIComponent(bookingId)}`, {
        method: 'DELETE'
      });
    } catch (e) {}

    showToast(`Booking ${bookingId} deleted.`);
  }

  // Table Event Delegation (Approve, Reject, Delete, View Proof)
  if (tableBody) {
    tableBody.addEventListener('click', (e) => {
      const actionBtn = e.target.closest('.btn-action-icon');
      if (actionBtn) {
        const action = actionBtn.getAttribute('data-action');
        const id = actionBtn.getAttribute('data-id');
        if (action === 'approve') {
```

```
          updateBookingStatus(id, 'Approved');
        } else if (action === 'reject') {
          updateBookingStatus(id, 'Rejected');
        } else if (action === 'delete') {
          deleteBooking(id);
        }
        return;
      }

      const proofBtn = e.target.closest('.proof-thumbnail-btn, .badge-cash');
      if (proofBtn) {
        const id = proofBtn.getAttribute('data-booking-id');
        openProofModal(id);
      }
... [Truncated for report brevity: 376 additional lines preserved in original project repository] ...
```

## VERBATIM SOURCE CODE — SCRIPT.JS (CUSTOMER BOOKING ENGINE & CHECKOUT CONTROLLER)

```
// ============================================
// L.S.G LAN CENTRE — JavaScript
// ============================================

document.addEventListener('DOMContentLoaded', () => {

  // ---------- Floating Particles ----------
  const canvas = document.getElementById('particles-canvas');
  const ctx = canvas.getContext('2d');
  let particles = [];
  let mouse = { x: null, y: null };

  function resizeCanvas() {
    canvas.width = window.innerWidth;
    canvas.height = window.innerHeight;
  }
  resizeCanvas();
  window.addEventListener('resize', resizeCanvas);

  class Particle {
    constructor() {
      this.reset();
    }

    reset() {
      this.x = Math.random() * canvas.width;
      this.y = Math.random() * canvas.height;
      this.size = Math.random() * 2 + 0.5;
      this.speedX = (Math.random() - 0.5) * 0.4;
      this.speedY = (Math.random() - 0.5) * 0.4;
      this.opacity = Math.random() * 0.4 + 0.1;
      this.hue = Math.random() > 0.5 ? 263 : 166; // purple or green
    }

    update() {
      this.x += this.speedX;
      this.y += this.speedY;

      // Mouse interaction
      if (mouse.x !== null) {
        const dx = mouse.x - this.x;
        const dy = mouse.y - this.y;
        const dist = Math.sqrt(dx * dx + dy * dy);
        if (dist < 150) {
          const force = (150 - dist) / 150;
          this.x -= (dx / dist) * force * 0.8;
```

```
        this.y -= (dy / dist) * force * 0.8;
      }
    }

    // Wrap around
    if (this.x < 0) this.x = canvas.width;
    if (this.x > canvas.width) this.x = 0;
    if (this.y < 0) this.y = canvas.height;
    if (this.y > canvas.height) this.y = 0;
  }

  draw() {
    ctx.beginPath();
    ctx.arc(this.x, this.y, this.size, 0, Math.PI * 2);
    ctx.fillStyle = `hsla(${this.hue}, 80%, 65%, ${this.opacity})`;
    ctx.fill();
  }
}

function initParticles() {
  const count = Math.min(80, Math.floor((canvas.width * canvas.height) / 15000));
  particles = [];
  for (let i = 0; i < count; i++) {
    particles.push(new Particle());
  }
}
initParticles();

function connectParticles() {
  for (let i = 0; i < particles.length; i++) {
    for (let j = i + 1; j < particles.length; j++) {
      const dx = particles[i].x - particles[j].x;
      const dy = particles[i].y - particles[j].y;
      const dist = Math.sqrt(dx * dx + dy * dy);
      if (dist < 120) {
        const opacity = (1 - dist / 120) * 0.15;
        ctx.beginPath();
        ctx.strokeStyle = `hsla(263, 80%, 65%, ${opacity})`;
        ctx.lineWidth = 0.5;
        ctx.moveTo(particles[i].x, particles[i].y);
        ctx.lineTo(particles[j].x, particles[j].y);
        ctx.stroke();
      }
    }
  }
}

function animateParticles() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  particles.forEach(p => {
    p.update();
    p.draw();
  });
  connectParticles();
  requestAnimationFrame(animateParticles);
}
animateParticles();

window.addEventListener('mousemove', (e) => {
  mouse.x = e.clientX;
  mouse.y = e.clientY;
});

window.addEventListener('mouseleave', () => {
  mouse.x = null;
  mouse.y = null;
});
```

```
// ---------- Navbar Scroll Effect ----------
const navbar = document.getElementById('navbar');
let lastScroll = 0;

window.addEventListener('scroll', () => {
  const currentScroll = window.scrollY;
  if (currentScroll > 50) {
    navbar.classList.add('scrolled');
  } else {
    navbar.classList.remove('scrolled');
  }
  lastScroll = currentScroll;
});


// ---------- Mobile Menu Toggle ----------
const mobileToggle = document.getElementById('mobileToggle');
const navLinks = document.getElementById('navLinks');

mobileToggle.addEventListener('click', () => {
  mobileToggle.classList.toggle('active');
  navLinks.classList.toggle('open');
});

// Close mobile menu when clicking a link
navLinks.querySelectorAll('a').forEach(link => {
  link.addEventListener('click', () => {
    mobileToggle.classList.remove('active');
    navLinks.classList.remove('open');
  });
});


// ---------- Counter Animation ----------
function animateCounters() {
  const counters = document.querySelectorAll('.hero-stat-value');
  counters.forEach(counter => {
    const target = parseInt(counter.getAttribute('data-count'));
    const duration = 2000;
    const startTime = performance.now();

    function updateCounter(currentTime) {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / duration, 1);
      const eased = 1 - Math.pow(1 - progress, 3); // ease-out cubic
      const current = Math.floor(eased * target);

      if (target >= 1000) {
        counter.textContent = current.toLocaleString() + '+';
      } else if (counter.closest('.hero-stat').querySelector('.hero-stat-
label').textContent.includes('AM')) {
        counter.textContent = current + ' AM';
      } else if (counter.closest('.hero-stat').querySelector('.hero-stat-
label').textContent.includes('Gbps')) {
        counter.textContent = current;
      } else {
        counter.textContent = current + '+';
      }

      if (progress < 1) {
        requestAnimationFrame(updateCounter);
      }
    }
```

```
    requestAnimationFrame(updateCounter);
  });
}

// Trigger counter animation when hero is in view
const heroObserver = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      animateCounters();
      heroObserver.unobserve(entry.target);
    }
  });
}, { threshold: 0.5 });

const heroStats = document.querySelector('.hero-stats');
if (heroStats) {
  heroObserver.observe(heroStats);
}



// ---------- Scroll Reveal ----------
const revealElements = document.querySelectorAll('.reveal');
const revealObserver = new IntersectionObserver((entries) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      entry.target.classList.add('visible');
      revealObserver.unobserve(entry.target);
    }
  });
}, {
  threshold: 0.1,
  rootMargin: '0px 0px -60px 0px'
});

revealElements.forEach(el => revealObserver.observe(el));



// ---------- Smooth Scroll for Anchor Links ----------
document.querySelectorAll('a[href^="#"]').forEach(anchor => {
  anchor.addEventListener('click', function (e) {
    e.preventDefault();
    const target = document.querySelector(this.getAttribute('href'));
    if (target) {
      target.scrollIntoView({ behavior: 'smooth' });
    }
  });
});



// ---------- Live Clock ----------
function updateClock() {
  const now = new Date();
  const options = {
    hour: '2-digit',
    minute: '2-digit',
    second: '2-digit',
    hour12: true,
    weekday: 'long'
  };
  const formatted = now.toLocaleTimeString('en-US', options);
  const dayName = now.toLocaleDateString('en-US', { weekday: 'long' });
  const clockEl = document.getElementById('currentTime');
  if (clockEl) {
    clockEl.textContent = `${dayName} — ${formatted}`;
  }
}
updateClock();
```

```
  setInterval(updateClock, 1000);


  // ---------- Contact Form ----------
  const contactForm = document.getElementById('contactForm');
  if (contactForm) {
    contactForm.addEventListener('submit', (e) => {
      e.preventDefault();

      const btn = contactForm.querySelector('button[type="submit"]');
      const originalText = btn.innerHTML;

      const contactName = document.getElementById('name')?.value || 'Inquiry';
      const contactPhone = document.getElementById('phone')?.value || '';
      const contactZone = document.getElementById('zone')?.value || 'General Inquiry';
      const contactMsg = document.getElementById('message')?.value || '';

      // Create booking/inquiry record for admin portal
      const inquiryRecord = {
        bookingId: `LSG-INQ-${Math.floor(1000 + Math.random() * 9000)}`,
        stationId: contactZone,
        stationName: `Inquiry: ${contactZone}`,
        date: new Date().toISOString().split('T')[0],
        slot: 'Message Inquiry',
        duration: 'N/A',
        totalAmount: '₹0',
        customer: {
          name: contactName,
          phone: contactPhone,
          email: 'N/A',
          discord: contactMsg
        },
        paymentStatus: 'Pending Verification',
        proof: null,
        createdAt: new Date().toISOString()
      };

      // Save to localStorage & broadcast
      try {
        const stored = JSON.parse(localStorage.getItem('lsg_reservations') || '[]');
        stored.unshift(inquiryRecord);
        localStorage.setItem('lsg_reservations', JSON.stringify(stored));

        const channel = new BroadcastChannel('lsg_live_sync');
        channel.postMessage({ type: 'NEW_BOOKING', booking: inquiryRecord });
      } catch (err) {}

      // Send to server API
      fetch('/api/bookings', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(inquiryRecord)
      }).catch(() => {});

      btn.innerHTML = `
        <svg width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor"
stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polyline points="20 6 9 17 4
12"/></svg>
        Message Sent to Admin!
      `;
      btn.style.background = 'linear-gradient(135deg, #06d6a0, #22d3ee)';

      setTimeout(() => {
        btn.innerHTML = originalText;
        btn.style.background = '';
        contactForm.reset();
      }, 3000);
```

```
    });
  }


  // ---------- Tilt Effect on Cards ----------
  const tiltCards = document.querySelectorAll('.zone-card, .pricing-card');
  tiltCards.forEach(card => {
    card.addEventListener('mousemove', (e) => {
      const rect = card.getBoundingClientRect();
      const x = e.clientX - rect.left;
      const y = e.clientY - rect.top;
      const centerX = rect.width / 2;
      const centerY = rect.height / 2;
      const rotateX = (y - centerY) / centerY * -3;
      const rotateY = (x - centerX) / centerX * 3;

      card.style.transform = `perspective(1000px) rotateX(${rotateX}deg) rotateY(${rotateY}deg)
translateY(-10px)`;
    });

    card.addEventListener('mouseleave', () => {
      card.style.transform = '';
    });
  });


  // ---------- Navbar Active Link Highlight ----------
  const sections = document.querySelectorAll('section[id]');
  const navLinksAll = document.querySelectorAll('.nav-links a:not(.nav-cta)');

  window.addEventListener('scroll', () => {
    let current = '';
    sections.forEach(section => {
      const sectionTop = section.offsetTop - 100;
      if (window.scrollY >= sectionTop) {
        current = section.getAttribute('id');
      }
    });

    navLinksAll.forEach(link => {
      link.style.color = '';
      if (link.getAttribute('href') === `#${current}`) {
        link.style.color = 'var(--accent-primary)';
      }
    });
  });


  // ---------- Parallax on Hero Image ----------
  const heroBg = document.querySelector('.hero-bg img');
  if (heroBg) {
    window.addEventListener('scroll', () => {
      const scrolled = window.scrollY;
      if (scrolled < window.innerHeight) {
        heroBg.style.transform = `translateY(${scrolled * 0.3}px) scale(1.1)`;
      }
    });
  }


  // ============================================
  // CHECKOUT & PAYMENT FLOW LOGIC
  // ============================================

  const STATIONS = {
    'battle-arena': {
```

```
    name: 'Gaming PC – Battle Arena',
    rate: 80,
    icon: '🖥️',
    spec: 'RTX 4090 • 240Hz • 1 Gbps Fiber'
  },
  'vip-lounge': {
    name: 'VIP Gaming PC – L.S.G Lounge',
    rate: 150,
    icon: '👑',
    spec: 'RTX 4090 • 360Hz • Soundproofed Pod'
  },
  'console-zone': {
    name: 'PS5 Console – Party Zone',
    rate: 100,
    icon: '🎮',
    spec: 'PS5 + Xbox • 65" 4K OLED • DualSense'
  },
  'racing-sim': {
    name: 'Racing Simulator Rig',
    rate: 120,
    icon: '🏎️',
    spec: 'Triple 144Hz • Direct Drive • Force Feedback'
  }
};

// State
let currentBooking = {
  stationId: 'battle-arena',
  date: '',
  slot: '10:00 AM - 12:00 PM',
  duration: 1,
  totalAmount: 80,
  paymentMethod: 'upi',
  customerName: '',
  customerPhone: '',
  customerEmail: 'N/A',
  customerDiscord: ''
};

let selectedProofFile = null;
let selectedProofDataUrl = null;

// DOM Elements - Checkout Form
const stationSelector = document.getElementById('stationSelector');
const bookingDateInput = document.getElementById('bookingDate');
const bookingSlotSelect = document.getElementById('bookingSlot');
const bookingDurationSelect = document.getElementById('bookingDuration');
const playerDiscordInput = document.getElementById('playerDiscord');
const checkoutNameInput = document.getElementById('checkoutName');
const checkoutPhoneInput = document.getElementById('checkoutPhone');
const checkoutEmailInput = document.getElementById('checkoutEmail');
const proceedToPaymentBtn = document.getElementById('proceedToPaymentBtn');
const checkoutValidationError = document.getElementById('checkoutValidationError');
const paymentMethodCards = document.querySelectorAll('.payment-method-card');

// DOM Elements - Order Summary
const sumStationIcon = document.getElementById('sumStationIcon');
const sumStationName = document.getElementById('sumStationName');
const sumStationSpec = document.getElementById('sumStationSpec');
const sumDate = document.getElementById('sumDate');
const sumSlot = document.getElementById('sumSlot');
const sumDuration = document.getElementById('sumDuration');
const sumRate = document.getElementById('sumRate');
const sumPaymentMethod = document.getElementById('sumPaymentMethod');
const sumSubtotal = document.getElementById('sumSubtotal');
const sumTotal = document.getElementById('sumTotal');
const btnPayAmount = document.getElementById('btnPayAmount');
const btnPayText = document.getElementById('btnPayText');
```

```
  const perkPaymentText = document.getElementById('perkPaymentText');

  // DOM Elements - Payment Modal
  const paymentModalOverlay = document.getElementById('paymentModalOverlay');
  const closePaymentModalBtn = document.getElementById('closePaymentModalBtn');
  const cancelPaymentBtn = document.getElementById('cancelPaymentBtn');
  const modalStepSubtitle = document.getElementById('modalStepSubtitle');
  const modalMethodTabs = document.querySelectorAll('.modal-method-tab');
  const modalTabUpi = document.getElementById('modalTabUpi');
  const modalTabCash = document.getElementById('modalTabCash');
  const modalStepsNav = document.getElementById('modalStepsNav');
  const stepIndicator1 = document.getElementById('stepIndicator1');
  const stepIndicator2 = document.getElementById('stepIndicator2');
  const stepIndicator3 = document.getElementById('stepIndicator3');
  const stepLine1 = document.getElementById('stepLine1');
  const stepLine2 = document.getElementById('stepLine2');

  // Modal Views
  const modalScanView = document.getElementById('modalScanView');
  const modalUploadView = document.getElementById('modalUploadView');
  const modalCashView = document.getElementById('modalCashView');
  const modalStatusView = document.getElementById('modalStatusView');

  // View 1 (Scanner)
  const modalPayableAmount = document.getElementById('modalPayableAmount');
  const modalBookingSummaryText = document.getElementById('modalBookingSummaryText');
  const copyUpiBtn = document.getElementById('copyUpiBtn');
  const upiIdText = document.getElementById('upiIdText');
  const completedPaymentBtn = document.getElementById('completedPaymentBtn');
  const switchToCashBannerBtn = document.getElementById('switchToCashBannerBtn');
  const bottomSwitchToCashBtn = document.getElementById('bottomSwitchToCashBtn');
  const switchBackToUpiBtn = document.getElementById('switchBackToUpiBtn');

  // View 2 (Upload Proof)
  const uploadStationText = document.getElementById('uploadStationText');
  const uploadAmountText = document.getElementById('uploadAmountText');
  const proofUploadZone = document.getElementById('proofUploadZone');
  const proofFileInput = document.getElementById('proofFileInput');
  const chooseFileBtn = document.getElementById('chooseFileBtn');
  const uploadEmptyState = document.getElementById('uploadEmptyState');
  const uploadPreviewState = document.getElementById('uploadPreviewState');
  const proofImgPreview = document.getElementById('proofImgPreview');
  const proofFileName = document.getElementById('proofFileName');
  const proofFileSize = document.getElementById('proofFileSize');
  const removeFileBtn = document.getElementById('removeFileBtn');
  const uploadFeedback = document.getElementById('uploadFeedback');
  const backToScannerBtn = document.getElementById('backToScannerBtn');
  const submitProofBtn = document.getElementById('submitProofBtn');
  const submitSpinner = document.getElementById('submitSpinner');
  const submitBtnText = document.getElementById('submitBtnText');

  // View Cash (Cash at Desk)
  const modalCashPayableAmount = document.getElementById('modalCashPayableAmount');
  const modalCashBookingSummaryText = document.getElementById('modalCashBookingSummaryText');
  const cancelCashBtn = document.getElementById('cancelCashBtn');
  const confirmCashBtn = document.getElementById('confirmCashBtn');
  const cashSpinner = document.getElementById('cashSpinner');
  const cashBtnText = document.getElementById('cashBtnText');
... [Truncated for report brevity: 867 additional lines preserved in original project
repository] ...
```

# 5. TESTING

The test suite was executed against the live L.S.G LAN CENTRE web application and administrative portal hosted on local and Wi-Fi LAN endpoints (http://localhost:3000/). The

test cases validate core operational scenarios including station selection, time slot restriction, UPI scanning, payment proof uploads, SSE broadcast latency, and admin status updates.

| Test ID | Test Case | Expected Result | Actual Result / Status |
|---|---|---|---|
| TC-01 | Open Homepage | Page loads with cyberpunk animations, navbar, and rig stats without blocking errors | Passed — Verified live on port 3000 |
| TC-02 | Station Zone Catalog | Hardware specifications, hourly rates, and features display correctly for all 4 zones | Passed — Verified |
| TC-03 | Time Slot Filtering | Booking slot dropdown only displays options between 10:00 AM and 08:00 PM | Passed — 5 discrete 2-hr slots active |
| TC-04 | Customer Input Validation | Submission without full name or valid phone number displays alert and prevents submission | Passed — Error banner triggers |
| TC-05 | UPI QR Scanner Modal | Clicking 'Proceed to Payment' opens modal with QR code and UPI ID paytm.s1h4qe4@pty | Passed — Modal renders correctly |
| TC-06 | Payment Proof Upload | Customer uploads transaction screenshot (<20MB); base64 preview generates cleanly | Passed — File accepted and encoded |
| TC-07 | Cash-on-Arrival Selection | Selecting Cash mode switches modal to Cash token instructions without requiring proof | Passed — Verified counter token |
| TC-08 | Real-Time SSE Broadcast | Submitted booking immediately broadcasts via SSE to all open Admin Portal tabs | Passed — Live sync < 200ms |
| TC-09 | Admin Dashboard KPI Update | Admin revenue, pending approvals, and occupancy KPIs update reactively upon new booking | Passed — KPI metrics accurate |
| TC-10 | Payment Verification Flow | Admin opens proof modal, inspects UPI screenshot, and clicks 'Approve'; status locks | Passed — Status changes to Approved |
| TC-11 | Walk-in Drawer Entry | Admin enters counter walk-in; record inserts at beginning and station capacity increments | Passed — Walk-in booked instantly |

| | | | |
|---|---|---|---|
| TC-12 | CSV Financial Export | Clicking 'Export CSV' triggers automatic download of sanitized spreadsheet with all booking fields | Passed — CSV file generated |

# 6. CONCLUSION

The L.S.G LAN CENTRE platform successfully demonstrates how modern web technologies can modernize and elevate gaming cafe operations. By replacing error-prone manual paper registers with an automated online reservation engine, transparent pricing, instant UPI mobile payments, and real-time administrative synchronization, the system eliminates station double-bookings and drastically reduces player wait times at the counter.

The implementation cleanly bifurcates customer discovery and administrative command. Gamers enjoy a visually captivating, responsive interface that enables them to reserve high-end RTX 4090 rigs and PS5 consoles with minimal friction, while reception staff maintain complete operational visibility via live Server-Sent Events, proof verification modals, and walk-in customer management.

## 6.1 Future Scope and Recommendations

- Automated UPI Webhook Reconciliation: Transition from manual screenshot proof verification to automated Payment Gateway webhooks (Razorpay / Cashfree / Paytm API) for instantaneous automatic approval.
- Hardware IoT Power Relays: Interface the web booking engine with networked smart power relays that automatically energize or power off specific gaming stations when a customer's reserved time slot begins or ends.
- Player Loyalty & Esports Tournament Leaderboard: Incorporate user accounts with XP points, hourly gaming discounts, and live tournament bracket matchmaking.
- Cloud Firestore & Firebase Hosting: Deploy the application across global CDN edge nodes with Cloud Firestore for multi-branch scalability across multiple city locations.

## 6.2 Final Conclusion

In conclusion, the development and documentation of the L.S.G LAN CENTRE system provides a comprehensive academic and practical demonstration of full-stack web engineering, real-time event streaming, asynchronous data flow, and modern cyber UI design. The project delivers an impactful digital solution tailored specifically to the operational requirements of Mumbai's thriving gaming community.

# 7. UNDERTAKING

I undertake that this project report entitled 'L.S.G LAN CENTRE — Live Web & Admin Portal' has been prepared for academic submission in partial fulfilment of the requirements for the award of the Degree of Bachelor of Science in Computer Science. All external

libraries, references, frameworks, and tools have been duly acknowledged. The implementation code, diagrams, schemas, and screenshots reflect genuine application development.

Student Name: Mihir Rajesh Kadam

College Name: SHETH L.U.J. & M.V. COLLEGE

Department: Department of Computer Science

Signature: _____ Date: _____

# 8. BIBLIOGRAPHY


- 1. Node.js Documentation, 'HTTP Server & Event Emitter Architecture', https://nodejs.org/docs/ (accessed October 2026).
- 2. Mozilla Developer Network (MDN), 'Using Server-Sent Events (SSE) with EventSource', https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events (accessed October 2026).
- 3. National Payments Corporation of India (NPCI), 'Unified Payments Interface (UPI) System Architecture & QR Code Specifications', https://www.npci.org.in/ (accessed October 2026).
- 4. OWASP Foundation, 'OWASP Top 10 Web Application Security Risks & File Upload Defenses', https://owasp.org/www-project-top-ten/ (accessed October 2026).
- 5. Google Firebase Documentation, 'Cloud Firestore & Firebase Realtime Database SDK', https://firebase.google.com/docs (accessed October 2026).
- 6. UML Consortium, 'Unified Modeling Language (UML) 2.5 Specification', Object Management Group (OMG), 2024.